

ADVANCED DATABASE SYSTEMS
ASSIGNMENT 1 — DESIGN OF A DISTRIBUTED DATABASE

CONTAINER TERMINAL & PORT LOGISTICS NETWORK

A four-site distributed database for New Mangalore, JNPT, Chennai and Kolkata

Sriram Venishetty

[Roll No.]

Department of Information Technology
National Institute of Technology Karnataka, Surathkal

Framework: Prof. Ananthanarayana V. S.

Contents

1. Problem Description.....	3
2. Data Sources.....	7
3. Queries and Access Frequencies.....	9
4. Entity Set.....	11
5. Relationships and Cardinalities.....	13
6. EER Diagram.....	14
7. Global Conceptual Schema.....	16
8. Normalization.....	18
9. Fragmentation.....	20
10. Physical Design.....	26
11. Replication and Allocation.....	31
12. Local Conceptual Schemas and Schema Integration.....	34
13. Work Area Space and System Specification.....	43
14. Conclusion.....	47
Appendix A — Amendments.....	48
Appendix B — Verification.....	48
Appendix C — Computation.....	49

1. Problem Description

1.1 Scope

A single container-terminal operator runs four Indian ports as one network. Each port is an autonomous operation — it berths vessels, discharges and loads containers, stacks them in its yard, clears them through customs, and releases them through road and rail gates. But the ports are not independent: containers are routinely transshipped, arriving on a mainline vessel at one port and departing on a feeder to another, and a single consignee typically has cargo moving through several ports at once.

The system must therefore let each port run on its own data at full speed, while still answering questions that only make sense across all four.

The ports also differ in what they do. Besides container handling, which all four share, each runs an operation the others do not have: liquid bulk at New Mangalore, on-dock rail and a bonded SEZ at JNPT, RoRo vehicle exports at Chennai, and tidal lighterage at Kolkata/Haldia. The design is therefore hybrid. A shared core is designed top-down for the container business (Sections 4–11); each port adds its own relations bottom-up for its own operation, so the four sites end up with four different local schemas, integrated through a global view (Section 12).

In scope (shared core): vessel arrivals and berthing, container discharge and loading, yard slot management, customs declaration and clearance, gate movements by road and rail, billing references (bill of lading, tariff, demurrage), consignees and shipping lines.

In scope (one site each): liquid bulk tank farm (S1), rail rakes and SEZ bonded transfers (S2), RoRo vehicles and inspections (S3), tidal windows and lighterage (S4) — Section 1.7.

Out of scope: dry bulk cargo, crane and equipment maintenance scheduling, employee payroll and rostering, vessel stowage planning, port security systems.

1.2 Data Flow

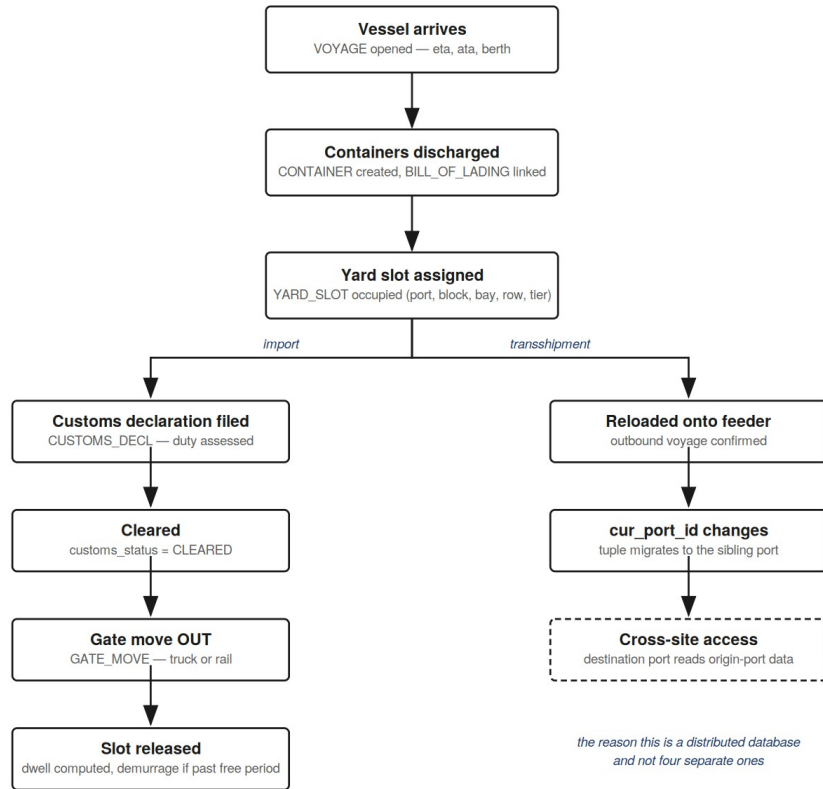


Figure 1 — Data flow: vessel arrival through gate-out, with the transshipment branch

The transshipment branch is the reason this is a distributed database and not four separate ones.

1.3 Control Flow

Actor	Triggers	Writes
Port control	Vessel arrival, berth allocation	VOYAGE
Ship planner	Discharge / load list per voyage	CONTAINER, BILL_OF_LADING
Yard planner	Slot assignment, yard shuffle	YARD_SLOT (U2)
Customs officer	Declaration filing, assessment, clearance	CUSTOMS_DECL (U3), CONTAINER.customs_status
Gate operator	Truck / rail entry and exit	GATE_MOVE (U1)
Line agent	Bill of lading issue, release order	BILL_OF_LADING
Finance	Demurrage and tariff computation	reads only

Table 1.1 — Control flow: who triggers what

Control is local-first: every write originates at the port where the physical event happens. Only reads cross site boundaries. This asymmetry is what the allocation analysis in Section 11 turns into a concrete placement decision.

1.4 Why this is an I/O-bound problem

GATE_MOVE is the dominant relation by two orders of magnitude — 60 million rows against 500,000 for the next largest. At 46 bytes per record that is roughly 2.76 GB, or 673,831 blocks at a 4 KB block size. An unindexed scan of it costs about 27 seconds of pure disk time.

The expensive queries — dwell time, yard occupancy, demurrage — are scans, joins and aggregates over that relation. None does meaningful computation per row. The cost is dominated by fetching blocks from disk and, where fragments are remote, moving them across the network. That is the definition of an I/O-bound workload, and it is why fragment placement, access-method choice and buffer sizing determine performance here rather than CPU.

Appendix B confirms the claim against the finished physical design: GATE_MOVE accounts for 96.8% of the core data blocks, and 94.4% once the site-specific relations of Section 12 are included.

1.5 Realistic Constraints

- A yard slot holds exactly one container; a container occupies at most one slot.
- Hazmat containers (hazmat_class not null) may not be stacked above tier 3.
- Reefer containers must be assigned slots with power connections.
- A container may not pass an OUT gate until customs_status = CLEARED.
- Free dwell period is 5 days from discharge; demurrage accrues per day thereafter.
- gross_wt must be at least tare_wt; the difference is billable net cargo weight.
- A container is physically present at exactly one port at a time (cur_port_id).
- Each port operates in its own local time; all stored timestamps are UTC.
- Customs data must remain at the port of clearance — a legal constraint on replication.

1.6 Sites

Site	Port	Character	Share of traffic
S1	New Mangalore (NMPT)	Regional, west coast	15%
S2	JNPT / Nhava Sheva	Largest container hub	40%
S3	Chennai	East coast hub	30%
S4	Kolkata / Haldia	River port, feeder traffic	15%

Table 1.2 — The four sites

Real inter-port distances are used rather than a uniform figure. Uniform distances make the allocation come out symmetric and the analysis says nothing. Propagation delay is $T_p = \text{distance} / v$ with $v = 2.7 \times 10^8$ m/s.

Link	Distance (km)	T_p (ms)	$2 \times T_p$ (ms)
S1 ↔ S2	700	2.59	5.19
S1 ↔ S3	700	2.59	5.19
S1 ↔ S4	2,000	7.41	14.81
S2 ↔ S3	1,300	4.81	9.63
S2 ↔ S4	1,900	7.04	14.07
S3 ↔ S4	1,650	6.11	12.22

Table 1.3 — Inter-site distances and propagation delays

1.7 Site-specific operations

Each port also runs one operation that exists nowhere else in the network. These are what make the four local schemas different, not merely four copies of one schema holding different rows.

Site	Local operation	Local relations	Local transactions
S1	Liquid bulk by pipeline into a tank farm	TANK_FARM, PIPELINE_TRANSFER	L1, L2
S2	On-dock rail to inland depots; SEZ bonded transfers	RAIL_RAKE, RAKE_LOADING, SEZ_UNIT, BOND_TRANSFER	L3, L4
S3	RoRo export of finished vehicles	RORO_VEHICLE, VEHICLE_INSPECTION	L5, L6
S4	Tidal draft windows; lighterage at anchorage	TIDAL_WINDOW, LIGHTERAGE	L7, L8

Table 1.4 — Site-specific operations (designed in Section 12)

2. Data Sources

The shared core is organised into six components, present at every site. Each owns a set of data and a set of operations.

Vessel and voyage management

Data: vessel_id, IMO number, flag, capacity, voyage, ETA/ATA/ATD, berth, status

Functionality: Records vessel calls, allocates berths, tracks arrival and departure against schedule.

Container management

Data: cont_id (ISO 6346), size, type, tare and gross weight, seal, hazmat class, current port, voyage

Functionality: Tracks every container in the network from discharge to final gate-out, including transshipment between ports.

Yard management

Data: port, block, bay, row, tier, occupying container, occupied since, block power and stack limits

Functionality: Assigns and releases slots, enforces reefer power and hazmat stacking rules, reports occupancy.

Customs management

Data: declaration id, HS code, duty amount, filed and cleared dates, officer

Functionality: Files declarations, assesses duty from the tariff schedule, records clearance, gates release.

Gate operations

Data: move id, container, port, timestamp, direction, truck, gate, operator

Functionality: Logs every road and rail movement in and out. The highest-volume component in the system.

Commercial management

Data: consignee, GSTIN, bill of lading, freight amount, service tariff and currency

Functionality: Links cargo to the paying party, prices port services, computes demurrage past the free period.

Four further components exist at one site each, owned and designed by that port:

Liquid bulk terminal (S1 only)

Data: tank, product HS code, capacity, current volume in kilolitres, measured density; pipeline transfers with volume and observed density

Functionality: Gauges tanks, records transfers between vessel and tank, reports custody in tonnes.

Rail and SEZ operations (S2 only)

Data: rake, train number, destination depot, wagons, departure; container-to-rake loading with rail weighment; SEZ units, bonds, bonded transfers

Functionality: Builds and despatches rakes, transfers containers into the SEZ under bond.

RoRo operations (S3 only)

Data: vehicle VIN, make, model, weight, deck, exporter, voyage; pre-shipment inspections

Functionality: Registers export vehicles, records damage surveys, releases vehicles to the RoRo voyage.

River access (S4 only)

Data: tidal window date, number, open and close time, maximum draft; lighterage operations by barge

Functionality: Publishes the draft timetable, lightens large ships at anchorage onto barges.

3. Queries and Access Frequencies

Nine retrieval queries and three update transactions against the global schema, plus eight local transactions that each run at one site only. The reads fall into three classes that touch largely disjoint attribute blocks of CONTAINER — the property that lets the fragmentation analysis in Section 9 produce a meaningful clustering rather than a near-uniform matrix.

A note on how this query set was chosen. In a production setting the workload is given and the design follows from it. Here the two were developed together: the queries reflect real terminal operations, but they were also chosen to exercise the techniques the framework requires — a workload whose attribute usage is uniform yields a degenerate affinity matrix and demonstrates nothing. What follows should be read as evidence that the method works on a realistic workload, not as the discovery of a structure latent in the data.

3.1 Simple queries

Q1 — Yard inventory for a block. List containers currently stacked in block B of port P, with slot position, size, type, gross weight and seal number.

Q2 — Hazmat stacking violation check. Find hazmat containers at port P occupying a slot above tier 3.

Q3 — Clearance backlog. Containers at port P with `customs_status` $\neq$ CLEARED whose declaration was filed more than seven days ago.

Q4 — Vessels at berth today. Vessels berthed at port P today with ETA, ATA, berth number and status.

3.2 Complex queries

Q5 — Yard occupancy in TEU per block. For port P, total TEU and total net cargo weight (`gross_wt` – `tare_wt`) per yard block.

Q6 — Total duty by consignee, network-wide. Total duty assessed for consignee C across all four ports. Cross-site.

Q7 — Discharge volume per voyage for a shipping line. Container count and total TEU discharged per voyage for line L in the last quarter.

Q8 — Demurrage exposure by consignee. For consignee C, containers past the five-day free period, with days overdue, declared value and demurrage payable at each port's tariff. Cross-site.

Q9 — Cargo in custody by port and class. Total tonnes currently in each port's custody, by cargo class. Cross-site, and heterogeneous: it reads the global relation `CARGO_IN_CUSTODY`, which each port computes from different local relations (Section 12.9).

3.3 Update transactions

These are what make cost computation possible in the allocation analysis. A design evaluated on reads alone reaches the opposite conclusion — Section 11.3 quantifies by how much.

U1 — Record a gate movement. Insert into `GATE_MOVE`; reads `CONTAINER` `seal_no` and `customs_status` to verify the seal and confirm clearance. Very high frequency, always local.

U2 — Reassign a yard slot. Update `YARD_SLOT` on a yard shuffle or restow; reads container size and hazmat class for the stacking rules.

U3 — Clear a customs declaration. Update `CUSTOMS_DECL`.`cleared_on` and `CONTAINER`.`customs_status`.

3.4 Access frequency matrix

Frequencies are deliberately unequal across sites, tracking each port's share of traffic. Uniform frequencies produce a symmetric allocation that demonstrates nothing.

Transaction	S1	S2	S3	S4	Total
Q1 Yard inventory	120	400	250	90	860
Q2 Hazmat check	30	90	60	20	200
Q3 Clearance backlog	40	150	100	35	325
Q4 Vessels at berth	25	60	45	20	150
Q5 Yard occupancy	60	180	120	50	410
Q6 Duty by consignee	15	50	30	10	105
Q7 Discharge per line	20	70	45	15	150
Q8 Demurrage exposure	25	80	55	20	180
Q9 Cargo in custody	5	10	5	5	25
U1 Gate move	3,000	11,000	7,000	2,200	23,200
U2 Slot reassign	400	1,500	900	300	3,100
U3 Customs clearance	350	1,300	850	280	2,780

Table 3.1 — Accesses per day, by site

U1 dominating by an order of magnitude is what will force the GATE_MOVE fragments to stay strictly local in Section 11 — the replication cost cannot be recovered by any read benefit.

3.5 Local transactions

Each site also runs transactions against its own local relations only. They never touch another site and are not visible to the global transaction manager; Section 12.10 gives their cost. They are listed here because two of them read or write core relations and therefore enter the global analysis: L3 closes a CONTAINER_C_2 row when a container is loaded onto a rake, and L5 reads CONSIGNEE_H to check a vehicle's exporter.

Transaction	Site	Per day
L1 Record tank gauging	S1	240
L2 Record pipeline transfer	S1	12
L3 Load container onto rake	S2	2,740
L4 Transfer container into SEZ	S2	110
L5 Register RoRo vehicle	S3	960
L6 Record vehicle inspection	S3	1,150
L7 Open / close lighterage operation	S4	6
L8 Publish tidal window	S4	2

Table 3.2 — Local transactions (5,220 a day in total)

4. Entity Set

4.1 Entity types

Entity	Type	Identifier	Note
PORT	Strong	port_id	Owner of two weak entities
SHIPPING_LINE	Strong	line_id	
VESSEL	Strong	vessel_id	imo_no is an alternate key
VOYAGE	Strong	voyage_id	One vessel calling at one port
CONSIGNEE	Strong	consignee_id	gstn is an alternate key
CONTAINER	Strong, specialized	cont_id	ISO 6346; superclass of four subtypes
YARD_SLOT	Weak	(port_id, block, bay, row, tier)	Owner PORT, via "located in"
GATE_MOVE	Strong	move_id	
CUSTOMS_DECL	Strong	decl_id	
BILL_OF_LADING	Strong	bl_no	
TARIFF	Weak	(port_id, service_code)	Owner PORT, via "applies at"

Table 4.1 — Entity types and identifiers

4.2 Extended features used

The framework asks for an EER diagram, not a plain ER diagram. Each extended feature below is used because the domain calls for it.

Specialization — CONTAINER. CONTAINER specializes into DRY, REEFER, TANK and OPEN_TOP. The constraint is disjoint and total: every container is exactly one type. Subtypes differ in attributes and, more importantly, in the yard rules they impose — reefers need powered slots, tanks carry a UN hazard number and a cleaning certificate, open-tops have an over-height that limits stacking.

Subtype	Additional attributes	Share	Tuples
DRY	none	87%	435,000
REEFER	set_point_temp, plug_id	8%	40,000
TANK	un_number, last_cleaned_on	3%	15,000
OPEN_TOP	over_height_cm	2%	10,000

Table 4.2 — CONTAINER subtypes

Weak entities — YARD_SLOT and TARIFF. A yard slot has no identity apart from its port: block A, bay 12, row 3, tier 2 means nothing until you say which port. Its partial key is the four-part position and its identifying relationship "located in" is drawn with a double diamond and total participation. TARIFF is weak for the same reason — a service code is only meaningful with the port whose rate card it belongs to.

Multivalued attribute — CONSIGNEE.phone. A consignee lists several contact numbers. Resolved into its own relation at 1NF.

Composite attribute — CONSIGNEE.address. Decomposes into street_line, area and pincode.

Derived attribute — CONTAINER.dwell_time. Gate-out timestamp minus discharge timestamp. Drawn with a dashed ellipse and not stored — it is recomputed. This matters in the physical design: the demurrage query must reconstruct the dwell start rather than read a stored column — Section 10.7 examines what that costs and what a stored discharge timestamp would save.

4.3 Foreign keys are not attributes

line_id, consignee_id, cur_port_id and voyage_id do not appear as attribute ellipses on CONTAINER. They arise from the "carried by", "consigned to", "currently at" and "discharges" relationships when the diagram is mapped to relations in Section 7. Showing a foreign key as both an ellipse and a relationship double-counts it.

4.4 Scope of the CONTAINER entity

CONTAINER represents a container's transit through the network — from first discharge to final gate-out — not the physical box across its whole service life. This is what makes "discharges" a 1:N relationship rather than M:N, and it is why the relation holds 500,000 rows (containers currently in the system) rather than every box the operator has ever handled.

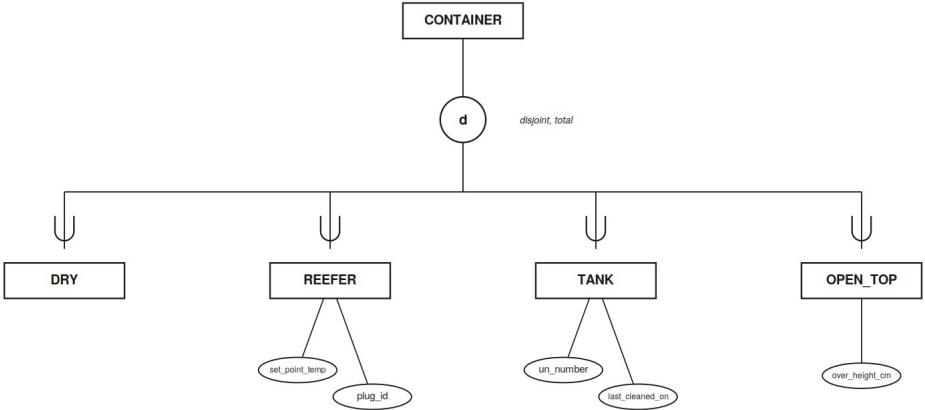
5. Relationships and Cardinalities

#	Entity 1	Relationship	Entity 2	Card.	Participation
1	SHIPPING_LINE	operates	VESSEL	1 : N	partial : total
2	VESSEL	makes	VOYAGE	1 : N	partial : total
3	VOYAGE	calls at	PORT	N : 1	total : partial
4	VOYAGE	discharges	CONTAINER	1 : N	partial : total
5	CONTAINER	carried by	SHIPPING_LINE	N : 1	total : partial
6	CONTAINER	consigned to	CONSIGNEE	N : 1	total : partial
7	CONTAINER	currently at	PORT	N : 1	total : partial
8	CONTAINER	occupies	YARD_SLOT	1 : 1	partial : partial
9	YARD_SLOT	located in (identifying)	PORT	N : 1	total : partial
10	CONTAINER	logs	GATE_MOVE	1 : N	partial : total
11	GATE_MOVE	recorded at	PORT	N : 1	total : partial
12	CONTAINER	declared in	CUSTOMS_DECL	1 : 1	partial : total
13	BILL_OF_LADING	issued for	VOYAGE	N : 1	total : partial
14	BILL_OF_LADING	billed to	CONSIGNEE	N : 1	total : partial
15	TARIFF	applies at (identifying)	PORT	N : 1	total : partial

Table 5.1 — Relationships, cardinalities and participation

There are no M:N relationships in the shared core. The scoping decision in Section 4.4 turns what would be a many-to-many container/voyage history into a one-to-many current-transit relationship. The practical consequence is that the mapping in Section 7 produces no junction relations — every relationship becomes a foreign key. (JNPT's local extension introduces the one M:N relationship in the design, containers loaded on rail rakes; it is modelled in Section 12.5.)

"occupies" is 1:1 with partial participation on both sides: a slot can be empty, and a container can be in transit between slots or already gated out.



DRY has no additional attributes; the subtypes differ in the constraints they impose on slot assignment.

Figure 3 — Specialization of CONTAINER (disjoint, total)

Figures 2 and 3 model the shared core. The entities that exist at one site only are modelled separately, per site, in Figure 6 (Section 12.5).

7. Global Conceptual Schema

The EER diagram is mapped to relations by the standard seven-step algorithm. Strong entities become relations; weak entities take the owner's key plus the partial key; 1:1 relationships place a foreign key with a UNIQUE constraint; 1:N relationships place a foreign key on the N side; there are no M:N relationships; the multivalued phone attribute becomes its own relation; and the specialization is mapped as a superclass relation plus one relation per subclass that has its own attributes.

Specialization mapping — justification. Keeping CONTAINER intact as a single relation matters downstream: the fragmentation analysis in Section 9 runs on it, and splitting the superclass across four relations would fracture that analysis for no benefit. It also avoids NULLs for the 87% of containers that are dry. `cont_type` serves as the discriminator, so a query never probes three relations to learn a container's type, and DRY needs no relation at all.

The schema below is the final form after normalization (Section 8). It is the global part of the design: every site holds its fragments of these relations. The ten relations that exist at one site only are defined in Section 12.6.

#	Relation	Attributes	Rec (B)	Tuples
1	PORT	port_id, port_name, city, country, timezone	70	4
2	SHIPPING_LINE	line_id, name, country, agent_name	84	120
3	VESSEL	vessel_id, name, imo_no, flag, capacity_teu, line_id	52	800
4	VOYAGE	voyage_id, vessel_id, port_id, eta, ata, atd, berth_no, status	50	3,600
5	CONSIGNEE	consignee_id, name, street_line, area, pincode, gstin	125	40,000
6	PINCODE	pincode, city, state	46	20,000
7	CONSIGNEE_PHONE	consignee_id, phone	16	60,000
8	CONTAINER	cont_id, size_ft, cont_type, tare_wt, gross_wt, seal_no, hazmat_class, invoice_value, customs_status, line_id, consignee_id, cur_port_id, voyage_id	76	500,000
9	YARD_BLOCK	port_id, block, has_power, max_tier	12	2,200
10	YARD_SLOT	port_id, block, bay, row, tier, cont_id, occupied_since	38	550,000
11	GATE_MOVE	move_id, cont_id, port_id, ts, direction, truck_no, gate_no, operator_id	46	60,000,000
12	CUSTOMS_DECL	decl_id, cont_id, hs_code, duty_amt, filed_on, cleared_on, officer_id	43	450,000
13	HS_CODE_MASTER	hs_code, hs_description, duty_rate	56	5,000
14	BILL_OF_LADING	bl_no, voyage_id, consignee_id, cont_count, freight_amt, issue_date	36	180,000
15	SERVICE	service_code, service_name, currency	39	10
16	TARIFF	port_id, service_code, rate	18	40
17	CONTAINER_REEFER	cont_id, set_point_temp, plug_id	23	40,000
18	CONTAINER_TANK	cont_id, un_number, last_cleaned_on	23	15,000
19	CONTAINER_OPEN_TO P	cont_id, over_height_cm	15	10,000

Table 7.1 — Global conceptual schema (19 relations in 3NF)

7.1 Primary and foreign keys

Relation	Primary key	Foreign key(s)
PORT	port_id	—
SHIPPING_LINE	line_id	—
VESSEL	vessel_id	line_id → SHIPPING_LINE
VOYAGE	voyage_id	vessel_id → VESSEL; port_id → PORT
CONSIGNEE	consignee_id	pincode → PINCODE
PINCODE	pincode	—

Relation	Primary key	Foreign key(s)
CONSIGNEE_PHONE	(consignee_id, phone)	consignee_id → CONSIGNEE
CONTAINER	cont_id	voyage_id → VOYAGE; line_id → SHIPPING_LINE; consignee_id → CONSIGNEE; cur_port_id → PORT
YARD_BLOCK	(port_id, block)	port_id → PORT
YARD_SLOT	(port_id, block, bay, row, tier)	(port_id, block) → YARD_BLOCK; cont_id → CONTAINER (UNIQUE)
GATE_MOVE	move_id	cont_id → CONTAINER (logical reference, not enforced); port_id → PORT
CUSTOMS_DECL	decl_id	cont_id → CONTAINER (UNIQUE); hs_code → HS_CODE_MASTER
HS_CODE_MASTER	hs_code	—
BILL_OF_LADING	bl_no	voyage_id → VOYAGE; consignee_id → CONSIGNEE
SERVICE	service_code	—
TARIFF	(port_id, service_code)	port_id → PORT; service_code → SERVICE
CONTAINER_REEFER / _TANK / _OPEN_TOP	cont_id	cont_id → CONTAINER

Table 7.2 — Primary and foreign keys

Why GATE_MOVE.cont_id is a logical reference. GATE_MOVE keeps a year of history — 60 million rows — while CONTAINER holds only the containers currently in the network (Section 4.4). When a container makes its final gate-out, its CONTAINER row is deleted but its gate moves must remain. A declared foreign key would either block that delete or cascade it into the history. The reference is therefore declared logical: U1 verifies that the container exists when it inserts a move, and no constraint is enforced afterwards. The same applies to the history relations of the JNPT extension (Section 12.6). This was found during the heterogeneous-schema work and is recorded as Amendment C3.

8. Normalization

This section normalizes the 19 global relations; the ten site-only relations go through the same procedure in Section 12.6. For each relation: state the functional dependencies, compute the closures that matter, identify candidate keys, then test 1NF → 2NF → 3NF in order. Only relations with a composite candidate key can violate 2NF — in this schema exactly three: YARD_SLOT, TARIFF and CONSIGNEE_PHONE. Every other relation reaches 2NF the moment it reaches 1NF.

8.1 Violations found and resolved

Relation	NF violated	Dependency	Resolution
CONSIGNEE	1NF	phone is multivalued	→ CONSIGNEE_PHONE
CONSIGNEE	1NF	address is composite	flattened to street_line, area, pincode
YARD_SLOT	2NF	(port_id, block) → has_power, max_tier	→ YARD_BLOCK
TARIFF	2NF	service_code → service_name, currency	→ SERVICE
VOYAGE	3NF	vessel_id → vessel_name	drop vessel_name
CONSIGNEE	3NF	pincode → city, state	→ PINCODE
CUSTOMS_DECL	3NF	hs_code → hs_description, duty_rate	→ HS_CODE_MASTER

Table 8.1 — Normalization violations

8.2 Worked example — YARD_SLOT (2NF)

Whether a slot has a power connection, and how high containers may be stacked in it, are properties of the block, not of the individual slot. As first drafted:

```
YARD_SLOT(port_id, block, bay, row, tier, has_power, max_tier, cont_id,
occupied_since)

(port_id, block, bay, row, tier) -> cont_id, occupied_since, has_power, max_tier
(port_id, block)                -> has_power, max_tier
```

{port_id, block, bay, row, tier}⁺ = all attributes, so it is the candidate key. (port_id, block) is a proper subset of that key and determines the non-prime attributes has_power and max_tier — a partial dependency, violating 2NF.

```
YARD_BLOCK(port_id, block, has_power, max_tier)          <- 3NF
YARD_SLOT(port_id, block, bay, row, tier, cont_id, occupied_since) <- 3NF
```

This decomposition is where two of the constraints in Section 1.5 become enforceable: has_power backs the reefer-slot rule, and max_tier backs the hazmat stacking limit that Q2 checks.

On cont_id: because "occupies" is 1:1, cont_id is unique across occupied slots. It is not a candidate key, because an empty slot has cont_id NULL and a candidate key cannot be null. It is implemented as a UNIQUE constraint.

8.3 Worked example — CUSTOMS_DECL (3NF)

```
CUSTOMS_DECL(decl_id, cont_id, hs_code, hs_description, duty_rate, duty_amt,
filed_on, cleared_on, officer_id)

decl_id -> cont_id, hs_code, hs_description, duty_rate, duty_amt, filed_on, ...
cont_id -> decl_id                      (declared in is 1:1)
hs_code -> hs_description, duty_rate     (the HS tariff schedule)

{decl_id}+ = all -> candidate key
{cont_id}+ = all -> alternate key
{hs_code}+ = hs_code, hs_description, duty_rate -> NOT a superkey
```

$\text{decl_id} \rightarrow \text{hs_code} \rightarrow \text{hs_description}, \text{duty_rate}$ is transitive, so the relation violates 3NF. The anomalies are real: a tariff revision would otherwise require updating every declaration carrying that heading, and a new HS code could not be recorded until a container was declared under it. `HS_CODE_MASTER` is extracted. `duty_amt` stays, because it is the assessed amount on that consignment, not a copy of the rate.

8.4 Dependencies tested and rejected

Three candidate dependencies were tested and found not to hold. Rejecting a plausible FD with a domain reason is stronger evidence of understanding than decomposing on one that merely looks true.

(size_ft, cont_type) → tare_wt in CONTAINER. Tempting, since ISO 6346 defines standard container classes. But tare weight is a measured per-unit value stencilled on each container, varying by manufacturer, repair history and flooring. Two 20-ft dry boxes routinely differ by 50–100 kg, and the difference is billable because net cargo weight is $\text{gross_wt} - \text{tare_wt}$. The dependency does not hold, so `tare_wt` stays in `CONTAINER` — which keeps the relation at 13 attributes for the fragmentation analysis.

gate_no → port_id in GATE_MOVE. Gates are numbered locally — every port has a Gate 1 — so `gate_no` alone does not determine the port. This is load-bearing rather than a technicality: `port_id` is the attribute used to fragment the largest relation in the schema. Had the FD held, fragmenting `GATE_MOVE` would have required a join on every access.

city → country in PORT. City names are not globally unique (Hyderabad exists in both India and Pakistan), so `PORT` is not decomposed on it.

8.5 Controlled redundancy

`CONTAINER.customs_status` duplicates information derivable from `CUSTOMS_DECL.cleared_on`. This is deliberate denormalization, not an oversight. U1 runs 11,000 times a day at S2 and must check clearance before releasing a container; storing the status on `CONTAINER` turns that check into a single-relation read. The cost is an extra write in U3, which runs 1,300 times a day — an order of magnitude less frequent. The trade is paid for by the frequency table.

8.6 Why stop at 3NF

The framework specifies 3NF, and the schema happens to satisfy BCNF as well. Scanning the final schema: in `CONSIGNEE` the determinants are `consignee_id` and `gstn`, both candidate keys; in `VESSEL` they are `vessel_id` and `imo_no`, both candidate keys; in `PINCODE`, `HS_CODE_MASTER` and `SERVICE` the only determinant is the primary key. The 3NF decomposition therefore cost nothing in lost dependency preservation. Going further would require multivalued or join dependencies, and the one multivalued fact in the design was already isolated into its own all-key relation at 1NF.

9. Fragmentation

9.1 Method, and one correction to the standard procedure

$\text{aff}(A_i, A_j)$ is the summed frequency, over all four sites, of every transaction that accesses both A_i and A_j . All twelve global transactions are included — nine retrievals and the three updates. Q9 enters because, although it is answered through the heterogeneous export views of Section 12, each site's branch reads `gross_wt` and `cur_port_id` of `CONTAINER_C` and `atd` of `VOYAGE`. The local transactions L1–L8 touch core relations only by key lookup or by whole-tuple delete (L3 closes a `CONTAINER_C_2` row), so they carry no attribute-affinity information and are excluded by the rule of Section 9.1.1; L5 does enter the allocation cost model (Section 11.5). U1 and U3 between them account for 25,980 of the 31,335 daily accesses to `CONTAINER`, so the fragmentation that follows is driven by the gate and clearance path rather than by the analytical queries.

The primary key must be excluded from the partitioning analysis. Running BEA on all 13 `CONTAINER` attributes produces $Z < 0$ at every split point — the best available score is $-22,500$. The reason is structural: `cont_id` is accessed by all eleven transactions that touch `CONTAINER`, so wherever the cut falls, `cont_id` lies on one side and every transaction needing it plus any attribute on the other side counts as an overlap query. COQ is therefore always large and either CTQ or CBQ is always zero, forcing $Z = (\text{CTQ} \times \text{CBQ}) - \text{COQ}^2 \leq 0$.

This is an artefact of including an attribute that is replicated into every fragment by construction. Vertical fragmentation must place the key in both fragments to make reconstruction possible, so the key cannot constrain the split. All analyses below run on the non-key attributes, with the key added back to each fragment afterwards.

9.1.1 Which transactions enter the affinity matrix

The twelve transactions above are not the only writes the database sees. Section 9.12 derives a container lifecycle load of roughly 41,000 inserts and 41,000 deletes a day network-wide, and Section 11.3 charges `CONTAINER_C` for it when computing replication cost. It is therefore reasonable to ask why that load does not also appear in the affinity matrix.

The rule applied here: whole-tuple operations — inserts and deletes — are excluded from the attribute affinity matrix, but included in the allocation cost model.

The justification is that an insert accesses every attribute of the tuple by definition, so it carries no information about which attributes belong together. It contributes the same affinity to every pair, which is the numerical equivalent of saying nothing. Its effect is not neutral, though: because it spans any cut, it lands in the overlap set at every split point and drives Z negative regardless of the data.

The magnitude is worth stating, because it shows the rule is load-bearing rather than a convenience:

Workload used for the affinity matrix	Best Z	Resulting split
Twelve transactions (as analysed here)	+106,932,375	{ <code>seal_no</code> , <code>customs_status</code> } vs the rest
Plus container lifecycle (82,000/day, all attributes)	-6,748,622,500	none — every split negative

Table 9.0 — Effect of admitting whole-tuple operations into the affinity matrix

Admitting them would make vertical fragmentation impossible for every relation in every design, not merely this one — which is the clearest evidence that the metric is being asked the wrong question. The cost model in Section 11 has no such problem: a replica genuinely must absorb every insert and delete, so they belong there in full.

State the rule before the matrices rather than after, and the two sections are consistent by construction.

9.2 Attribute usage matrix — CONTAINER

	A2	A3	A4	A5	A6	A7	A8	A9	A10	A11	A12	A13	freq
Q1	1	1	0	1	1	0	0	0	0	0	1	0	860
Q2	0	0	0	0	0	1	0	0	0	0	1	0	200
Q3	0	0	0	0	0	0	0	0	0	1	1	0	325
Q5	1	0	1	1	0	0	0	0	0	0	1	0	410
Q6	0	0	0	0	0	0	0	1	0	0	1	0	105
Q7	1	0	0	0	0	0	1	0	0	0	1	1	150
Q8	0	0	0	0	0	0	0	1	1	0	1	0	180
Q9	0	0	0	1	0	0	0	0	0	0	1	0	25
U1	0	0	0	0	1	0	0	0	0	1	0	0	23,200
U2	1	0	0	0	0	1	0	0	0	0	0	0	3,100
U3	0	0	0	0	0	0	0	0	0	1	0	0	2,780

Table 9.1 — Attribute usage matrix. A2 size_ft · A3 cont_type · A4 tare_wt · A5 gross_wt · A6 seal_no · A7 hazmat_class · A8 line_id · A9 consignee_id · A10 invoice_value · A11 customs_status · A12 cur_port_id · A13 voyage_id

9.3 Attribute affinity matrix

	A2	A3	A4	A5	A6	A7	A8	A9	A10	A11	A12	A13
A2	4520	860	410	1270	860	3100	150	0	0	0	1420	150
A3	860	860	0	860	860	0	0	0	0	0	860	0
A4	410	0	410	410	0	0	0	0	0	0	410	0
A5	1270	860	410	1295	860	0	0	0	0	0	1295	0
A6	860	860	0	860	24060	0	0	0	0	23200	860	0
A7	3100	0	0	0	0	3300	0	0	0	0	200	0
A8	150	0	0	0	0	0	150	0	0	0	150	150
A9	0	0	0	0	0	0	0	285	180	0	285	0
A10	0	0	0	0	0	0	0	180	180	0	180	0
A11	0	0	0	0	23200	0	0	0	0	26305	325	0
A12	1420	860	410	1295	860	200	150	285	180	325	2255	150
A13	150	0	0	0	0	0	150	0	0	0	150	150

Table 9.2 — Attribute affinity matrix

The $\text{aff}(A6, A11) = 23,200$ cell dominates everything else by an order of magnitude. That single number is the whole story of this fragmentation: the gate check reads seal_no and customs_status together, 23,200 times a day.

9.4 BEA clustering and split quality

Applying $\text{cont}(A_i, A_k, A_j) = 2 \cdot \text{bond}(A_i, A_k) + 2 \cdot \text{bond}(A_k, A_j) - 2 \cdot \text{bond}(A_i, A_j)$, placing one column at a time at its position of maximum contribution, yields the clustered order:

A10 A9 A4 A5 A7 A2 A12 A6 A11 A3 A13 A8

A11 placing next to A6 with a contribution of 2,330,658,000 — three orders of magnitude above any other placement — is the algorithm recovering the gate-check pair. Evaluating $Z = (\text{CTQ} \times \text{CBQ}) - \text{COQ}^2$ at each cut point:

Cut	TA (left of cut)	CTQ	CBQ	COQ	Z
1	A10	0	31,155	180	-32,400
2	A10,A9	0	31,050	285	-81,225
3	+A4	0	30,640	695	-483,025

Cut	TA (left of cut)	CTQ	CBQ	COQ	Z
4	+A5	0	29,755	1,580	-2,496,400
5	+A7	0	26,455	4,880	-23,814,400
6	+A2	3,100	26,305	1,930	77,820,600
7	+A12	4,020	25,980	1,335	102,657,375
8	+A6	4,020	2,780	24,535	-590,790,625
9	+A11	30,325	0	1,010	-1,020,100
10	+A3	31,185	0	150	-22,500
11	+A13	31,185	0	150	-22,500

Table 9.3 — Split quality at each cut point. BEA result: cut 7, Z = 102,657,375

The collapse at cut 8 is instructive — it separates A6 from A11, which puts U1 into the overlap set and costs roughly 690 million in Z.

9.5 Exhaustive verification

BEA evaluates only contiguous cuts of the clustered order, so it searches 11 of the 4,094 possible binary partitions. Evaluating all of them shows BEA's answer ranks 5th, 4.0% below optimum:

	Partition	Z
Optimum	F_a = {seal_no, customs_status} • F_b = the other ten	106,932,375
BEA	F_a = {size_ft, tare_wt, gross_wt, hazmat_class, consignee_id, invoice_value, cur_port_id}	102,657,375

Table 9.4 — BEA result against the exhaustive optimum

The optimum is also far more interpretable: it isolates exactly the two attributes the gate check reads. Three other partitions tie for it, differing only in where the low-affinity attributes land; among the tied optima the one with the smallest hot fragment is chosen, since that fragment is read 25,980 times a day.

9.6 Adopted vertical fragmentation of CONTAINER

Fragment	Attributes	Record	Read by	Freq/day
CONTAINER_G (gate & clearance)	cont_id, seal_no, customs_status	33 B	U1, U3	25,980
CONTAINER_C (cargo, yard & commercial)	cont_id, size_ft, cont_type, tare_wt, gross_wt, hazmat_class, line_id, consignee_id, invoice_value, cur_port_id, voyage_id	54 B	Q2, Q5–Q9, U2	4,170
both	—	—	Q1, Q3	1,185

Table 9.5 — Adopted vertical fragmentation

$$Z = 25,980 \times 4,170 - 1,185^2 = 108,336,600 - 1,404,225 = 106,932,375$$

Storage: 33 + 54 = 87 B against 76 B unfragmented — 11 B (14.5%) of key-replication overhead, the price of reconstruction. The payoff: the highest-frequency transaction in the entire workload now reads a 33-byte fragment instead of a 76-byte record — 57% less data per access, 23,200 times a day.

9.7 The other relations — computed, then excluded

Every candidate relation was put through the same procedure. Only CONTAINER yields a beneficial split.

Relation	Best Z	Verdict
CONTAINER	+106,932,375	Fragment
CUSTOMS_DECL	0	Do not fragment — hs_code and officer_id are cold

Relation	Best Z	Verdict
VOYAGE	7,500	Do not fragment — the only positive split isolates atd for Q9 (25/day) and saves about 11 ms a day
CONSIGNEE	0	Fragment anyway, on replication grounds (below)
GATE_MOVE	-538,240,000	Do not fragment

Table 9.6 — Vertical fragmentation candidates

GATE_MOVE is the strongest exclusion. Every split scores between -538 and -547 million. The affinity matrix is almost uniform — every cell lies between 23,200 and 23,380, because U1 writes all eight attributes together. There is no attribute grouping to discover. Splitting the largest relation in the schema would add a reconstruction join to the highest-frequency write in the workload and buy nothing.

CONSIGNEE: Z = 0 does not mean "do not fragment". It means the metric cannot see the benefit here, because Z measures the cost of queries spanning fragments and no query reads the cold attributes at all. CONSIGNEE has no port_id, so it cannot be fragmented horizontally, yet Q6 and Q8 read it from all four sites. Replicating a 44-byte hot fragment (consignee_id, name) instead of the whole 125-byte relation cuts replicated volume by 65%.

9.8 Horizontal fragmentation

The simple predicates are the four port memberships. Since a tuple belongs to exactly one port, the minterm predicates reduce to the four simple ones and the fragmentation is complete and disjoint by construction.

p1: port = S1 (New Mangalore) p3: port = S3 (Chennai)
p2: port = S2 (JNPT) p4: port = S4 (Kolkata/Haldia)

Relation	Predicate attribute	Fragments
CONTAINER_C	cur_port_id	CONTAINER_C_k = $\sigma(\text{cur_port_id} = k)(\text{CONTAINER_C})$
VOYAGE	port_id	VOYAGE_k = $\sigma(\text{port_id} = k)(\text{VOYAGE})$
YARD_SLOT	port_id	YARD_SLOT_k = $\sigma(\text{port_id} = k)(\text{YARD_SLOT})$
YARD_BLOCK	port_id	YARD_BLOCK_k = $\sigma(\text{port_id} = k)(\text{YARD_BLOCK})$
GATE_MOVE	port_id	GATE_MOVE_k = $\sigma(\text{port_id} = k)(\text{GATE_MOVE})$
TARIFF	port_id	TARIFF_k = $\sigma(\text{port_id} = k)(\text{TARIFF})$

Table 9.7 — Primary horizontal fragmentation

Why GATE_MOVE fragments on its own port_id rather than derivedly from CONTAINER. Both are available and they differ: a transshipped container's moves are spread across ports, so fragmenting by GATE_MOVE.port_id keeps each move where it physically happened, while deriving from CONTAINER would gather a container's whole history at its current port. Deriving would help Q8 (180/day) and hurt U1 (23,200/day), which must write locally at the gate. U1 outweighs Q8 by 129:1, so primary fragmentation on port_id wins.

9.9 Derived horizontal fragmentation

Six relations carry no port attribute and are fragmented by semijoin with an already-fragmented parent:

CONTAINER_G_k	= CONTAINER_G	x CONTAINER_C_k	(on cont_id)
CUSTOMS_DECL_k	= CUSTOMS_DECL	x CONTAINER_C_k	(on cont_id)
CONTAINER_REEFER_k	= CONTAINER_REEFER	x CONTAINER_C_k	(on cont_id)
CONTAINER_TANK_k	= CONTAINER_TANK	x CONTAINER_C_k	(on cont_id)
CONTAINER_OPEN_TOP_k	= CONTAINER_OPEN_TOP	x CONTAINER_C_k	(on cont_id)
BILL_OF_LADING_k	= BILL_OF_LADING	x VOYAGE_k	(on voyage_id)

Note the first line: the vertical split put `cur_port_id` in `CONTAINER_C`, so `CONTAINER_G` has no port attribute of its own and must be derived from its sibling. The vertical and horizontal decisions interact — that is what makes this mixed fragmentation rather than two independent steps.

9.10 Mixed fragmentation tree

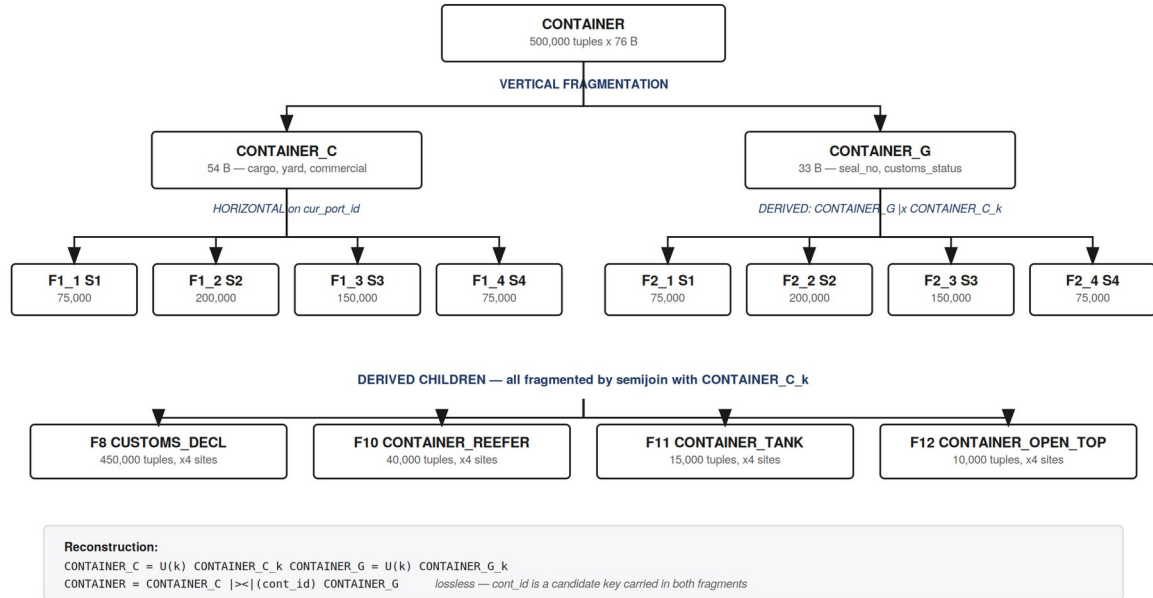


Figure 4 — Mixed fragmentation tree for `CONTAINER`

9.11 Correctness of the fragmentation

Completeness. The two vertical fragments' attribute sets union to all 13 attributes. The four horizontal predicates are exhaustive because `cur_port_id` and `port_id` are NOT NULL foreign keys to `PORT`, which has exactly four rows. Every `CUSTOMS_DECL` row has a non-null `cont_id` (1:1 with `CONTAINER`) and every container sits in exactly one `CONTAINER_C_k`, so every declaration lands in exactly one fragment.

The completeness of derived fragmentation depends on referential integrity: if a `CUSTOMS_DECL` row could carry a `cont_id` with no matching container, it would vanish from every fragment. The foreign-key constraint is what makes the fragmentation correct.

Reconstruction.

```
CONTAINER_C = CONTAINER_C_1 U CONTAINER_C_2 U CONTAINER_C_3 U CONTAINER_C_4
CONTAINER_G = CONTAINER_G_1 U CONTAINER_G_2 U CONTAINER_G_3 U CONTAINER_G_4
CONTAINER   = CONTAINER_C |><| (cont_id) CONTAINER_G

GATE_MOVE   = U_k GATE_MOVE_k          CUSTOMS_DECL = U_k CUSTOMS_DECL_k
YARD_SLOT   = U_k YARD_SLOT_k          CONSIGNEE    = CONSIGNEE_H |><| CONSIGNEE_R
```

The vertical join is lossless because `cont_id` is a candidate key of `CONTAINER` and is carried in both fragments: the decomposition satisfies $F_a \cap F_b = \{cont_id\}$ and $\{cont_id\} \rightarrow F_a$. That condition is what makes the key replication necessary rather than merely convenient.

Disjointness. Horizontal: the four predicates are mutually exclusive — a container has exactly one `cur_port_id`. Derived: inherited from the parent, since the semijoin attribute is a foreign key to a single parent tuple. Vertical: disjoint except for the replicated key, the standard and necessary exception.

9.12 Fragment migration

CONTAINER_C is fragmented on `cur_port_id`, which changes when a container is transshipped — so tuples migrate between fragments, taking their derived children with them. Quantifying it:

annual container transits	= 60,000,000 / 4 moves	= 15,000,000
daily transits	= 15,000,000 / 365	= 41,096
cross-check: 500,000 resident	/ 41,096 per day	= 12.2 days average dwell
at a 15% transshipment rate		= 6,160 migrations/day

Each migration moves one CONTAINER_C tuple (54 B), its CONTAINER_G tuple (33 B), its CUSTOMS_DECL row (43 B) and any subtype row — under 150 B, as one delete plus one insert across sites. Against 29,080 daily updates, migration adds roughly 21% to the write load. That is real and should not be described as negligible.

The alternative is to fragment on the immutable discharge port, so no migration ever occurs. The cost: a container physically at S2 but discharged at S1 would live in S1's fragment, so every yard query (Q1, Q2, Q5 — 1,470/day) would have to union fragments from all four sites instead of reading one locally. That is 4,410 remote query executions a day, each returning a multi-tuple result set, against 6,160 fixed 150-byte migrations. Migration is adopted.

10. Physical Design

10.1 Assumptions and derived parameters

Parameter	Value	Parameter	Value
Block size (B)	4,096 B	Average seek (S)	8 ms
Inter-block gap (g)	128 B	Average latency (L)	4 ms
Block pointer	8 B	Disk transfer rate	100 MB/s
Record organisation	spanned, fixed	Packet size	1,024 B
Integer / Char / Decimal	4 / 1 / 8 B	Bandwidth (WAN)	100 Mbps
Date / Datetime	4 / 8 B	Propagation speed	2.7×10^8 m/s

Table 10.1 — Frozen physical assumptions

All four derived constants are computed rather than asserted:

$$\begin{aligned}
 Tr &= (B + g) / \text{transfer rate} = (4096 + 128) / 104,857,600 = 0.0403 \text{ ms} \\
 Td &= \text{packet} / \text{bandwidth} = 1024 / 12,500,000 = 0.0819 \text{ ms per packet} \\
 Tp &= \text{distance} / v \quad (\text{see Table 1.3}) \\
 \text{one random block access} &= S + L + Tr = 12.04 \text{ ms}
 \end{aligned}$$

Access-time formulas — the sequential and scattered cases must be distinguished, because a scan of GATE_MOVE at S2 costs 10.9 s sequentially but would cost 54 minutes if every block cost a seek:

$$\begin{aligned}
 \text{sequential scan of } N \text{ contiguous blocks} &: S + L + (N \times Tr) && [\text{one seek}] \\
 N \text{ scattered accesses (index probes)} &: (S + L + Tr) \times N \\
 \text{local update of } N \text{ blocks} &: 2 \times (S + L + Tr) \times N \\
 \text{remote retrieval} &: \text{local} + Td_{\text{total}} + 2Tp \\
 \text{remote update} &: \text{local update} + 2Tp
 \end{aligned}$$

10.2 Storage requirements

Records are spanned and fixed-length, so blocks = $\lceil n \times R / B \rceil$ with no internal fragmentation.

ID	Fragment	R (B)	bfr	S1	S2	S3	S4	Total
F1	CONTAINER_C	54	75	989	2,637	1,978	989	6,593
F2	CONTAINER_G	33	124	605	1,612	1,209	605	4,031
F3	VOYAGE	50	81	7	18	14	7	46
F4	YARD_SLOT	38	107	766	2,042	1,531	766	5,105
F5	YARD_BLOCK	12	341	1	3	2	1	7
F6	GATE_MOVE	46	89	101,075	269,532	202,149	101,075	673,831
F7	TARIFF	18	227	1	1	1	1	4
F8	CUSTOMS_DECL	43	95	709	1,890	1,418	709	4,726
F9	BILL_OF_LADING	36	113	238	633	475	238	1,584
F10	CONTAINER_REEFER	23	178	34	90	68	34	226
F11	CONTAINER_TANK	23	178	13	34	26	13	86
F12	CONTAINER_OPEN_TOP	15	273	6	15	11	6	38
	Per-site total			104,444	278,507	208,882	104,444	696,277

Table 10.2 — Blocks per fragment per site

GATE_MOVE alone is 673,831 of 696,277 core blocks — 96.8%, and still 94.4% once the 17,400 blocks of site-only relations (Section 12.11) are added. This is the I/O-bound claim of Section 1.4 restated at block level against the finished design.

ID	Unfragmented relation	R (B)	Tuples	Blocks
G1	PORT	70	4	1
G2	SHIPPING_LINE	84	120	3
G3	VESSEL	52	800	11
G4	HS_CODE_MASTER	56	5,000	69
G5	SERVICE	39	10	1
G6	PINCODE	46	20,000	225
G7	CONSIGNEE_H	44	40,000	430
G8	CONSIGNEE_R	85	40,000	831
G9	CONSIGNEE_PHONE	16	60,000	235
	Total (one copy)			1,806

Table 10.3 — Unfragmented relations

10.3 Access method comparison — GATE_MOVE at S2

24,000,000 records · 269,532 data blocks · 1,104 MB · 6,000,000 distinct cont_id values. Index entry = key size + 8-byte block pointer.

Method	Index blocks	Size	Accesses	Verdict
(a) Heap file, no index	0	0	269,532	Baseline — 10.9 s per scan
(b) Primary index on move_id (3 levels)	795	3 MB	4	Useless — no query searches by move_id
(c) Clustering index on cont_id (3 levels)	28,038	115 MB	4	Cheapest index, but conflicts with U1
(d) Secondary index on truck_no (4 levels)	106,197	435 MB	5	Useless — no query searches by truck_no
(e) B+ tree on cont_id over heap (h = 4)	112,149	459 MB	5	CHOSEN

Table 10.4 — Access method comparison

Why (b) and (d) are rejected. An index is only worth its space if a query searches on its key. Nothing in the workload searches GATE_MOVE by move_id or truck_no.

Why (c) is rejected despite being four times smaller. A clustering index requires the file physically ordered by cont_id. But U1 inserts 11,000 gate moves a day at S2, arriving in timestamp order, not container order. Maintaining cont_id order under that insert rate means either overflow chains — which degrade the very lookups the index exists to serve — or periodic reorganisation of a 1.1 GB file. Choosing an access method that fights the dominant write pattern is a false economy.

Why (e) is chosen. The heap file stays append-only, which is exactly what U1 wants. The B+ tree provides the lookup Q8 needs, and B+ tree inserts are the operation B+ trees are designed for.

10.4 Justification by the numbers

full scan = $S + L + N \times Tr = 8 + 4 + 269,532 \times 0.0403 = 10,870 \text{ ms}$ (10.9 s)
 B+ tree lookup (worst case) = $4 \text{ index} + 1 \text{ data} = 5 \times 12.04 = 60.20 \text{ ms}$
 speed-up = 181x

Buffering halves it again. The B+ tree's non-leaf levels at S2 total $517 + 3 + 1 = 521$ blocks = 2.13 MB — small enough to stay permanently resident. Only the leaf access reaches disk, so a lookup costs 1 leaf + 1 data = 24.08 ms. 2.13 MB of buffer converts a 60 ms lookup into a 24 ms lookup, permanently.

Whether the index pays is a question with an answer, not an assumption:

	Cost
U1 insert with index (non-leaf cached): leaf read + leaf write + data write	36.12 ms
U1 insert without index: read last block + write	24.08 ms

	Cost
Extra per insert × 23,200/day = index maintenance	279 s/day
Q8 without index: scan F6 at all four sites, 180/day	4,895 s/day
Q8 with index: ~16 containers × (1 leaf + 1 data), 180/day	87 s/day
Net saving 4,808 s/day for 279 s/day cost	17× return

Table 10.5 — Index break-even analysis

The index is justified, but by a factor of 17 rather than by an order of magnitude — and it costs 459 MB, a 42% storage overhead on the fragment. If U1's frequency were five times higher, or Q8's five times lower, the decision would reverse.

10.5 Chosen access method per fragment

ID	Access method	S1	S2	S3	S4	Accesses
F1	primary index on cont_id	6	14	11	6	3
F2	primary index on cont_id	4	9	7	4	3
F3	primary index on voyage_id	1	1	1	1	2
F4	primary index on (block, bay, row, tier)	7	15	12	7	2
F5, F7	none — single block	0	0	0	0	1
F6	B+ tree on cont_id over heap	42,056	112,149	84,112	42,056	4
F8	primary index on cont_id	5	10	8	5	3
F9	primary index on bl_no	3	5	4	3	3
F10–F12	primary index on cont_id	1	1	1	1	2
	Index blocks total	42,085	112,206	84,158	42,085	

Table 10.6 — Index blocks per fragment per site

F5 YARD_BLOCK and F7 TARIFF get no index at all: both occupy a single block, so an index would cost an extra access to save none. F4 YARD_SLOT is ordered on its primary key, which makes Q1 ("all containers in block B") a contiguous range scan rather than a filtered full scan — the key order was chosen to match the query.

10.6 Fragment access times

ID	Fragment	S1	S2	S3	S4
F1	CONTAINER_C	51.8	118.2	91.7	51.8
F2	CONTAINER_G	36.4	76.9	60.7	36.4
F3	VOYAGE	12.3	12.7	12.6	12.3
F4	YARD_SLOT	42.9	94.3	73.7	42.9
F5	YARD_BLOCK	12.0	12.1	12.1	12.0
F6	GATE_MOVE	4,083.6	10,869.6	8,155.2	4,083.6
F7	TARIFF	12.0	12.0	12.0	12.0
F8	CUSTOMS_DECL	40.6	88.1	69.1	40.6
F9	BILL_OF_LADING	21.6	37.5	31.1	21.6
F10–F12	subtype fragments	≤ 13.4	≤ 15.6	≤ 14.7	≤ 13.4

Table 10.7 — Full-fragment scan time at the home site (ms)

Every fragment except F6 scans in under 120 ms. F6 is two orders of magnitude slower than anything else, which is why it is the only fragment whose access method required real analysis.

10.7 A cheaper alternative the schema does not offer

The index above is justified against the query as posed. It is worth asking the prior question: why does Q8 need GATE_MOVE at all?

Q8 needs one fact per container — when its dwell began. The schema offers two sources and neither is satisfactory. YARD_SLOT.occupied_since is cheap, but U2 (slot reassign, 3,100/day) resets it, so any re-stowed container loses its original dwell start and the query would return wrong answers. GATE_MOVE with direction = IN is correct and immutable, but it is 60 million rows.

The schema has no immutable discharge timestamp. That omission, rather than any property of the workload, is what forces the demurrage query onto the largest relation in the database and justifies a 1,148 MB index.

Storing discharge_ts (8 B, written once at discharge, never updated) on CONTAINER_C would remove the need entirely:

	Cost
CONTAINER_C record	54 B → 62 B
Extra data blocks, all sites	+977 (4.0 MB)
GATE_MOVE B+ tree, all sites — no longer required by any query	1,148 MB
Index maintenance removed	279 s/day
Database size	4,091 MB → 2,946 MB (28% smaller)

Table 10.9 — Cost of a stored discharge timestamp against the index it would replace

4 MB of data would replace 1,148 MB of index — 287× cheaper — and Q8 would get faster too, since it already scans CONTAINER_C to find the consignee's containers and would then need nothing else.

The design retains GATE_MOVE and its index for one reason worth stating: GATE_MOVE remains the authoritative record for containers that have already left the yard, where no current CONTAINER row exists, and for any historical demurrage audit. A production design would store discharge_ts for the live query path and keep GATE_MOVE for history, taking both benefits. The figures above are the honest cost of not doing so.

This is recorded rather than silently corrected because the access-method analysis in Sections 10.3 to 10.6 is a valid answer to the question it was asked. The point is that optimising an access path is not the same as asking whether the schema should have made the query cheap in the first place.

10.8 Remote access — the finding that shapes allocation

Route	Local	+ Td	+ 2Tp	Total	Ratio to local
S1 → S2	36.12	0.082	5.19	41.39 ms	1.15×
S2 → S3	36.12	0.082	9.63	45.83 ms	1.27×
S2 → S4	36.12	0.082	14.07	50.28 ms	1.39×
S1 → S4	36.12	0.082	14.81	51.02 ms	1.41×

Table 10.8 — Remote single-record retrieval of the gate fragment

A remote read costs only 1.15× to 1.41× a local read. Disk access (36 ms) dominates network latency (5–15 ms), and transmission delay is negligible for small results. This is the most consequential number in the physical design: replication is far less attractive here than intuition suggests, because the benefit of placing a fragment locally is at most a 41% saving on read time while every replica multiplies the cost of every update to it.

It is also a consequence of the assumptions. With an 8 ms seek the disk is a spinning drive; on SSDs (≈ 0.1 ms access) the local read would fall well under a millisecond and the remote/local ratio would exceed 50 $\times$, flipping the conclusion entirely.

11. Replication and Allocation

11.1 Method

All Beneficial Sites. For each fragment F and candidate site S , place a copy at S if the read benefit exceeds the update cost.

$$\text{Benefit}(F, S) = \text{SUM over read transactions } q \text{ at } S \text{ that access } F: \\ \text{freq}(q, S) \times (\text{remote cost} - \text{local cost}) \\ \text{where remote} - \text{local} = Td_{\text{total}} + 2Tp(S, \text{home})$$

$$\text{Cost}(F, S) = \text{SUM over update transactions } u \text{ that write } F \text{ at its home site:} \\ \text{freq}(u) \times (\text{local update cost} + 2Tp(\text{home}, S))$$

Allocate a copy at $S \iff \text{Benefit}(F, S) > \text{Cost}(F, S)$

11.2 Two structural facts

Of the twelve global transactions, nine never leave the site where they originate. Only Q6, Q8 and Q9 span sites, and together they run 310 times a day. The 5,220 local transactions of Section 12.10 never leave their site by construction.

	Per day
Transactions served entirely locally	31,175
Cross-site transactions (Q6, Q8, Q9)	310
Remote fragment accesses they cause	930
Locality of global accesses: 1 – 930 / 32,415	97.13%
Locality including local transactions L1–L8	97.53%

Table 11.1 — Locality of the base allocation

From the Section 10.7 timings, the break-even follows directly:

$$\begin{aligned} \text{a local replica SAVES, per read} &: Td + 2Tp &= 5.27 \dots 14.90 \text{ ms} \\ \text{a local replica COSTS, per update} &: \text{local_update} + 2Tp &= 77.43 \dots 87.06 \text{ ms} \end{aligned}$$

$\Rightarrow$ a fragment must be read 6 to 15 times more often than it is updated, at that specific site, before a replica breaks even.

11.3 Allocation of the fragmented relations

Only F1, F6 and F8 are read across sites, so only they are candidates. Worst case shown: home site S2, which carries 40% of the workload.

Fragment	Writes/day	Replica at	Benefit (ms/day)	Cost (ms/day)	B – C	Decision
F1 CONTAINER_C	88,160	S1	237	2,730,322	-2,730,085	no
		S3	874	2,887,051	-2,886,177	no
		S4	495	3,043,780	-3,043,284	no
F6 GATE_MOVE	23,200	S1	132	718,506	-718,374	no
		S3	534	759,750	-759,216	no
		S4	283	800,995	-800,711	no
F8 CUSTOMS_DECL	43,780	S1	79	1,355,870	-1,355,791	no
		S3	291	1,433,701	-1,433,410	no
		S4	142	1,511,532	-1,511,391	no

Table 11.2 — Benefit and cost for the cross-site fragments

Every fragment F1–F12 is allocated non-redundantly, one copy at the site named by its predicate. Not one is read often enough from elsewhere to justify the write amplification.

11.4 What benefit-only computation would have concluded

Delete the Cost column and every one of the nine decisions above reverses — F1, F6 and F8 would be replicated to all four sites, because their benefit is positive everywhere.

Fragment	Benefit (ms/day)	Cost (ms/day)	Cost : Benefit
F1 CONTAINER_C	1,606	8,661,153	5,393 : 1
F6 GATE_MOVE	949	2,279,251	2,402 : 1
F8 CUSTOMS_DECL	512	4,301,103	8,401 : 1
Total	3,067	15,241,507	4,970 : 1

Table 11.3 — The cost of omitting cost computation

In plain terms: the benefit-only allocation would spend 254 minutes a day of extra update time to save 3.1 seconds a day of read time, and would grow the database from 2.8 GB to 11.3 GB. The cost term is not a formality — it is the term that decides the answer.

11.5 Allocation of the unfragmented relations

Relation	Blocks	Read by	Updates/day	Decision
PORT	1	Q1–Q5, Q7, Q9	0.01	Replicate to all 4
SHIPPING_LINE	3	Q7	0.1	Replicate to all 4
VESSEL	11	Q4, Q7	1.0	Replicate to all 4
SERVICE	1	Q8	0.01	Replicate to all 4
CONSIGNEE_H	430	Q6, Q8, L5	20	S2 + replica at S3
HS_CODE_MASTER	69	—	0.2	Single copy at S2
PINCODE	225	—	0.1	Single copy at S2
CONSIGNEE_R	831	—	20	Single copy at S2
CONSIGNEE_PHONE	235	—	15	Single copy at S2

Table 11.4 — Allocation of the unfragmented relations

Four relations pass, and for the same reason: they are tiny, read from every site, and effectively never updated. Together they occupy 16 blocks — 64 KB — so replicating them four times costs 256 KB against a 4 GB database. The method identifies the textbook case for full replication without being told.

Four relations have no read demand at all in the frozen workload, which traces straight back to the cold attributes identified in Section 9.7. Benefit is zero at every site by definition. This is a statement about the twelve global transactions this design is optimised for, not about the data being useless — a customs audit or a mailing run would read them.

CONSIGNEE_H was the one close call, and the local workload settles it. On the global transactions alone, a replica at S3 loses by a factor of about 2 (benefit 825 ms/day against 1,637 ms/day of cost). But S3's local transaction L5 checks every RoRo vehicle's exporter against CONSIGNEE_H — 960 reads a day that would otherwise all be remote. Counting them, S3 reads CONSIGNEE_H 1,045 times a day, the benefit rises to 10,148.6 ms/day against the same 1,637.4 ms/day of cost, and a replica at S3 is placed (Amendment C2). S1 and S4, with no such local demand, still lose by a factor of 4–7. The case is instructive: a site's own schema generated workload that changed a global allocation decision.

11.6 Final allocation

Site	Fragments held	Replicated relations
S1	F1-F12 for New Mangalore	PORT, SHIPPING_LINE, VESSEL, SERVICE
S2	F1-F12 for JNPT	the four above, plus CONSIGNEE_H and single copies of HS_CODE_MASTER, PINCODE, CONSIGNEE_R, CONSIGNEE_PHONE
S3	F1-F12 for Chennai	PORT, SHIPPING_LINE, VESSEL, SERVICE, CONSIGNEE_H
S4	F1-F12 for Kolkata/Haldia	PORT, SHIPPING_LINE, VESSEL, SERVICE

Table 11.5 — Final allocation

Each site additionally holds its own site-only relations (Section 12), which are never replicated. Replication adds 478 blocks network-wide — 48 for the four small relations and 430 for the CONSIGNEE_H copy at S3, 0.05% of total storage. A design that looked like it might be replication-heavy turns out to be almost entirely partitioned, and the numbers say why.

11.7 What the cost model does not capture

All Beneficial Sites optimises performance. It says nothing about availability, and the result has a consequence worth naming: with one copy of every fragment, losing S2 makes 40% of the database unavailable — every JNPT gate move, clearance and yard query stops, and Q6, Q8 and Q9 cannot complete anywhere because they aggregate across all four ports.

This is not a flaw in the computation; it is a different objective. The correct treatment is to separate the two. The performance allocation stands as computed. For availability, provide an asynchronous standby replica of each site's fragments at a partner site ($S1 \leftrightarrow S2$, $S3 \leftrightarrow S4$, pairing the shortest links). A standby serves no reads, so it contributes no benefit term, and it is updated outside the transaction, so it contributes no synchronous cost term — it is invisible to the All Beneficial Sites calculation while still bounding data loss. Storage impact: fragment storage doubles to roughly 8 GB, which is the honest price of durability.

12. Local Conceptual Schemas and Schema Integration

Sections 4 to 11 design a shared core top-down: one global schema, fragmented and allocated across four sites. On its own that produces four databases with the same structure, differing only in which rows they hold. A distributed database in the full sense is more than that — it is a set of databases at different locations whose schemas themselves differ, because the organisations running them do different things. This section adds that dimension. Each port gets relations that exist at that port and nowhere else, the four resulting Local Conceptual Schemas are pairwise different, and the global layer integrates them through a mapping rather than by forcing them into one shape.

12.1 Why the four sites need different schemas

The four ports share container handling, which is why the core exists. Each also runs an operation that the others do not have at all:

Site	Operation found only here	What it must record	Native unit
S1 New Mangalore	Liquid bulk: crude and POL cargo discharged by pipeline into a tank farm	Tanks, their contents and density; each pipeline transfer	kilolitres
S2 JNPT	On-dock rail to inland container depots; transfers into the port's SEZ under bond	Rakes and the containers loaded on them; SEZ units and bonded transfers	rail weighment, kg
S3 Chennai	RoRo export of finished vehicles	Each vehicle by VIN; pre-shipment damage inspections	vehicles, kg
S4 Kolkata / Haldia	Riverine access: tidal draft windows and lighterage of large ships at anchorage	Published tidal windows; each barge operation	tonnes

Table 12.1 — Site-specific operations

None of this belongs in the global schema. Putting TANK_FARM there would give three sites an empty relation they must still create, secure and back up; putting all four operations there would make every port's schema the union of everyone's business, and every new local operation a change to the global schema that all four sites must agree to. That is the opposite of design autonomy. The operations are local, so their schemas are local.

12.2 Design approach: top-down core, bottom-up extensions

The design is therefore hybrid. The core (Sections 4–11) is designed top-down from the network-wide requirements. Each port's extension is designed bottom-up from that port's own operations, and integrated afterwards. Two rules keep the two halves from interfering:

R1 — an extension never alters a core relation. Site-specific facts go into new relations that reference core keys (voyage_id, cont_id, consignee_id); no column is ever added to a core relation at one site only. The core therefore stays identical at every site, Q1–Q8 and U1–U3 run unchanged everywhere, and nothing in Sections 7–11 is invalidated.

R2 — the global layer sees an extension only through that site's export schema. A site publishes what the network needs to know about its local relations as a view over them, in agreed units. The global layer never names TANK_FARM or RORO_VEHICLE; it names the export schema. A port can restructure its extension freely as long as its export view still produces the same columns.

12.3 Schema architecture

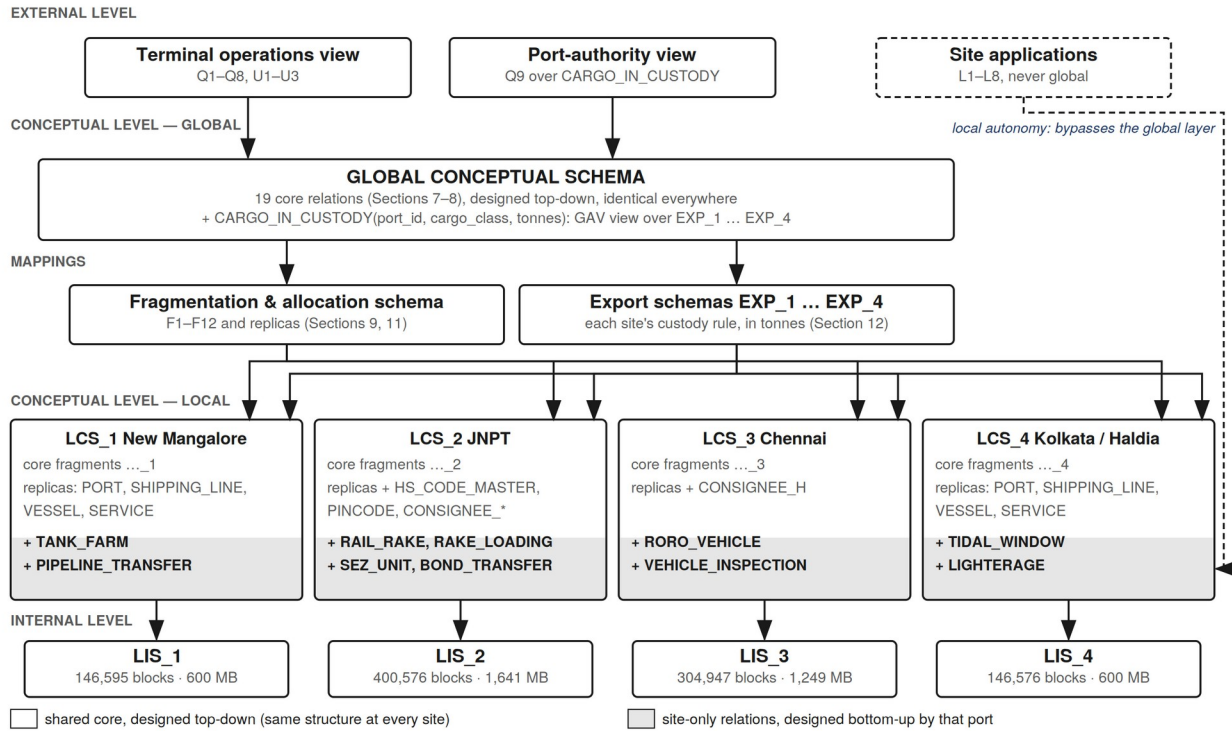


Figure 5 — Three-level schema architecture with heterogeneous local schemas

The figure is the standard reference architecture for a distributed database, populated with this design. External schemas are the views users work through. The Global Conceptual Schema holds the 19 core relations and one integrated relation, CARGO_IN_CUSTODY, defined over the four export schemas. Two mappings connect it to the sites: the fragmentation and allocation schema of Sections 9 and 11 for the core, and the export schemas of Section 12.7 for the extensions. Each site has its own Local Conceptual Schema and its own Local Internal Schema. Local applications (L1–L8, Section 12.10) bypass the global layer entirely.

12.4 The four Local Conceptual Schemas

	Core fragments	Replicated / single-copy relations	Site-only relations	Relations
LCS_1 New Mangalore	12 (..._1)	PORT, SHIPPING_LINE, VESSEL, SERVICE	TANK_FARM, PIPELINE_TRANSFER	18
LCS_2 JNPT	12 (..._2)	the four above + HS_CODE_MASTER, PINCODE, CONSIGNEE_H, CONSIGNEE_R, CONSIGNEE_PHONE	RAIL_RAKE, RAKE_LOADING, SEZ_UNIT, BOND_TRANSFER	25
LCS_3 Chennai	12 (..._3)	the four above + CONSIGNEE_H	RORO_VEHICLE, VEHICLE_INSPECTION	19
LCS_4 Kolkata / Haldia	12 (..._4)	PORT, SHIPPING_LINE, VESSEL, SERVICE	TIDAL_WINDOW, LIGHTERAGE	18

Table 12.2 — Composition of each Local Conceptual Schema

No two LCSs are equal, and no site-only relation appears in more than one of them. The difference is not only in the extensions: even the core part differs, because allocation placed the unfragmented relations unevenly (S2 is home to the commercial master data; S3 holds a CONSIGNEE_H replica for a reason given in Section 12.10). The 12 core fragment families are the only thing all four share, and they share only structure — no row of a core fragment exists at more than one site.

12.5 Local EER extensions

Figure 6 shows the entities each port adds, drawn per site. Grey boxes are core entities from Figure 2, shown only where a local entity attaches to them.

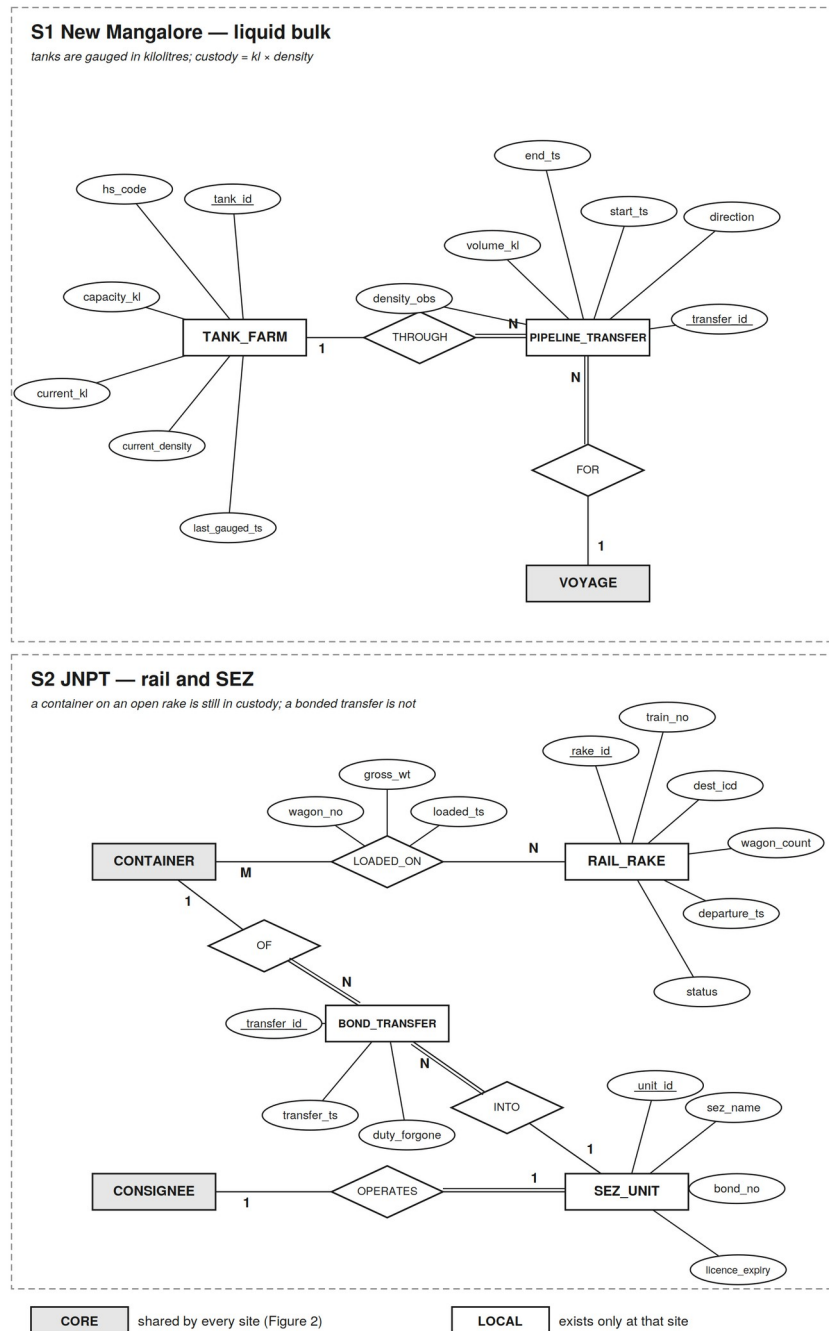


Figure 6a — Site-specific EER extensions: S1 New Mangalore and S2 JNPT

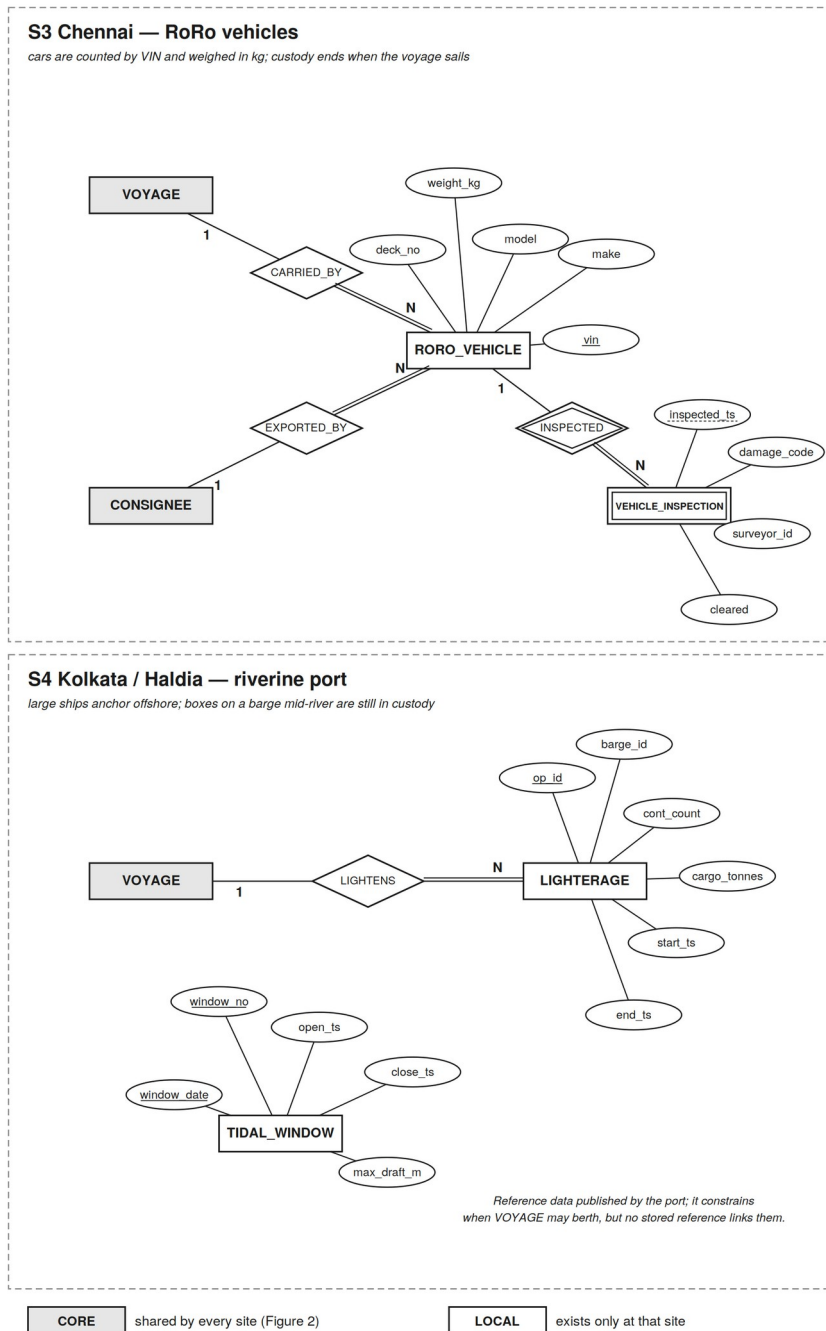


Figure 6b — Site-specific EER extensions: S3 Chennai and S4 Kolkata / Haldia

Each port models its own operation with the same EER constructs the core uses, independently of the others. JNPT's LOADED_ON is the only many-to-many relationship in the whole design: a rake carries many containers, and over the year of history retained a container can travel on more than one rake. It maps to the junction relation RAKE_LOADING. Chennai's VEHICLE_INSPECTION is a weak entity — an inspection has no identity apart from the vehicle it examined — identified by vin plus the partial key inspected_ts. Haldia's TIDAL_WINDOW stands alone: it constrains when a VOYAGE may berth, but as a published timetable rather than a stored reference.

12.6 Local relations and their normalization

#	Site	Relation	Attributes (key in first position)	Rec (B)	Tuples
R20	S1	TANK_FARM	tank_id, hs_code, capacity_kl, current_kl, current_density, last_gauged_ts	44	60

#	Site	Relation	Attributes (key in first position)	Rec (B)	Tuples
R21	S1	PIPELINE_TRANSFER	transfer_id, tank_id, voyage_id, direction, start_ts, end_ts, volume_kl, density_obs	45	4,400
R22	S2	RAIL_RAKE	rake_id, train_no, dest_icd, wagon_count, departure_ts, status	40	11,000
R23	S2	RAKE_LOADING	(rake_id, cont_id), wagon_no, gross_wt, loaded_ts	31	1,000,000
R24	S2	SEZ_UNIT	unit_id, consignee_id (UNIQUE), sez_name, bond_no (UNIQUE), licence_expiry	57	200
R25	S2	BOND_TRANSFER	transfer_id, cont_id, unit_id, transfer_ts, duty_forgone	35	40,000
R26	S3	RORO_VEHICLE	vin, voyage_id, exporter_id, make, model, weight_kg, deck_no	68	350,000
R27	S3	VEHICLE_INSPECTION	(vin, inspected_ts), damage_code, surveyor_id, cleared	34	420,000
R28	S4	TIDAL_WINDOW	(window_date, window_no), open_ts (UNIQUE), close_ts, max_draft_m	32	730
R29	S4	LIGHTERAGE	op_id, voyage_id, barge_id, cont_count, cargo_tonnes, start_ts, end_ts	44	2,200

Table 12.3 — Site-only relations (3NF)

The same procedure as Section 8 was applied to each local relation. Two violations were found in first drafts; three plausible dependencies were rejected.

BOND_TRANSFER (3NF). The first draft carried bond_no. Each SEZ unit operates under one bond, so transfer_id → unit_id → bond_no is transitive. bond_no was removed; it lives once in SEZ_UNIT, where it is an alternate key.

TANK_FARM (3NF). The first draft carried the product name alongside its HS code. hs_code → product description is the dependency already resolved by HS_CODE_MASTER in Section 8.3, so only hs_code is kept.

RAKE_LOADING (2NF test). The key is (rake_id, cont_id). Does cont_id alone determine gross_wt? It does not: gross_wt here is the rail weighbridge reading for this loading, and over a year of history the same container can be loaded, weighed and despatched more than once. Every non-key attribute depends on the whole key, so the relation is in 2NF — and, with no transitive dependency, in 3NF.

Rejected: hs_code → current_density in TANK_FARM. Density of the same grade varies by batch and temperature, and custody tonnes are computed from the measured value. It is a measurement, as tare_wt is in Section 8.4.

Rejected: model → make, and (make, model) → weight_kg in RORO_VEHICLE. Model names are not unique across manufacturers, and weight varies by variant and fitment. Both stay as stored attributes of the vehicle.

All ten relations are in 3NF, and in BCNF: every determinant is a candidate key.

References to the core. Every local reference to a core relation resolves at the same site, which is what makes it enforceable locally: PIPELINE_TRANSFER.voyage_id → VOYAGE_1, RORO_VEHICLE.voyage_id → VOYAGE_3, LIGHTERAGE.voyage_id → VOYAGE_4, SEZ_UNIT.consignee_id → CONSIGNEE_H at S2, and RORO_VEHICLE.exporter_id → the CONSIGNEE_H replica at S3. Three are declared as logical references instead, for the same reason as GATE_MOVE in Table 7.2: RAKE_LOADING.cont_id and BOND_TRANSFER.cont_id record history that outlives the container's row in CONTAINER_C, and TANK_FARM.hs_code points to HS_CODE_MASTER, which is held only at S2. They are checked by the transaction that writes them, not enforced as constraints.

12.7 Export schemas and the global relation

The network needs one fact from the extensions: how much cargo each port holds in custody right now. Every site exports it in one uniform shape, export_custody(port_id, cargo_class, tonnes), but each computes it from different relations by a different rule:

```
-- S1 New Mangalore: tanks gauged in kilolitres; kl x density = tonnes
SELECT 1, 'LIQUID_BULK', SUM(current_kl * current_density) FROM tank_farm
```

```

-- S2 JNPT: a container on a rake that has not departed is still in custody
SELECT 2, 'CONTAINER_ON_RAIL', SUM(rl.gross_wt) / 1000.0
  FROM rake_loading rl JOIN rail_rake r ON r.rake_id = rl.rake_id
 WHERE r.departure_ts IS NULL

-- S3 Chennai: vehicles aboard a voyage that has not sailed
SELECT 3, 'RORO', SUM(v.weight_kg) / 1000.0
  FROM roro_vehicle v JOIN voyage y ON y.voyage_id = v.voyage_id
 WHERE y.atd IS NULL

-- S4 Kolkata/Haldia: containers on a barge that has not landed
SELECT 4, 'CONTAINER_AFLOAT', SUM(cargo_tonnes) FROM lighterage WHERE end_ts IS NULL

-- every site also exports its core containers:
SELECT k, 'CONTAINER', SUM(gross_wt) / 1000.0 FROM container_c      -- CONTAINER_C_k

-- global relation (Global-As-View):
CARGO_IN_CUSTODY = EXP_1 UNION ALL EXP_2 UNION ALL EXP_3 UNION ALL EXP_4

```

This is Global-As-View integration: the global relation is defined as a query over the sources, so a global query is answered by substituting that definition and sending each branch to its site. The alternative, Local-As-View, defines each source as a view over the global schema and needs query rewriting to answer anything. GAV is chosen because the sources are few, known and stable, and it keeps query processing to plain view expansion. Its known cost is that adding a fifth port means redefining CARGO_IN_CUSTODY — acceptable here, since adding a port also adds a fragment to every core relation.

12.8 Semantic heterogeneity

The export views are short, but the difficulty they solve is not syntactic. "Cargo in custody" means something different at each port, and a naive integration double-counts or loses cargo at exactly the boundaries where the ports differ.

Site	Custody begins	Custody ends	Conversion to tonnes
S1	Pipeline transfer into a tank	Pipeline transfer out	kl × measured density
S2	Discharge into the yard	The rake departs — not when the box leaves the yard for the rail siding	rail weight kg ÷ 1000
S3	Vehicle registered for export	The carrying voyage sails (atd set)	manifest kg ÷ 1000
S4	Barge leaves the anchored ship	Voyage departs — a box on a barge mid-river is still the port's	barge manifest, already tonnes

Table 12.4 — Four definitions of custody

The JNPT row shows why the mapping must be defined by the site. When a container is loaded onto a rake (L3), its CONTAINER_C_2 row closes — from the yard's point of view it has left. A global view that counted only CONTAINER_C would lose it for the hours before the rake departs. S2's export view picks it up from RAKE_LOADING instead, so the total is conserved across the hand-over and drops only when the rake actually leaves. Section 12.13 tests exactly this.

12.9 Q9 — a global query over four different schemas

Q9 — Cargo in custody by port and class. For each port and cargo class, total tonnes currently in the port's custody. Issued by the port authority 25 times a day (S1 5, S2 10, S3 5, S4 5). It reads CARGO_IN_CUSTODY, so each site answers it with a different local plan:

Site	Local sub-query	Plan	Time (ms)
S1	CONTAINER_C_1 + TANK_FARM	two scans (989 + 1 blocks)	63.9
S2	CONTAINER_C_2 + RAIL_RAKE ⋈ RAKE_LOADING	scan rakes, clustering index to open rakes	171.0
S3	CONTAINER_C_3 + VOYAGE_3 ⋈ RORO_VEHICLE	three scans (1,978 + 14 + 5,811 blocks)	350.3

Site	Local sub-query	Plan	Time (ms)
S4	CONTAINER_C_4 + LIGHTERAGE	two scans (989 + 24 blocks)	64.8

Table 12.5 — Q9 sub-queries

Run in parallel, Q9 completes in 355–368 ms depending on the originating site (the slowest part plus a round trip and a 5 ms merge); run serially, 675–691 ms. The slowest part is S3's scan of RORO_VEHICLE — a local relation the global optimiser cannot see. Only the export schema is visible to it, and S3 chooses its own plan. That is execution autonomy in practice.

Q9 also reads core attributes: gross_wt and cur_port_id of CONTAINER_C, and atd of VOYAGE. It therefore enters the affinity analysis of Section 9. Re-running it with Q9 included leaves the CONTAINER split unchanged (the optimum rises to $Z = 106,932,375$), and gives VOYAGE a best split of $Z = 7,500$ that isolates atd for Q9 alone — a saving of about 11 ms a day, not worth a second fragment. VOYAGE stays whole.

12.10 Local transactions and site autonomy

#	Site	Transaction	Per day	ms
L1	S1	Record tank gauging (UPDATE TANK_FARM)	240	24.1
L2	S1	Record pipeline transfer (INSERT PIPELINE_TRANSFER)	12	24.1
L3	S2	Load container onto rake (INSERT RAKE_LOADING; close CONTAINER_C_2 row)	2,740	60.2
L4	S2	Transfer container into SEZ unit (read SEZ_UNIT; INSERT BOND_TRANSFER)	110	36.2
L5	S3	Register RoRo vehicle (check exporter in CONSIGNEE_H; INSERT RORO_VEHICLE)	960	108.4
L6	S3	Record vehicle inspection (probe RORO_VEHICLE by vin; INSERT)	1,150	60.2
L7	S4	Open or close a lighterage operation	6	48.2
L8	S4	Publish a tidal window	2	24.1
		Total	5,220	

Table 12.6 — Local transactions

All three forms of site autonomy hold for these transactions. Design autonomy: each port defined its extension without any change to another site or to the core. Communication autonomy: none of L1–L8 sends a message to another site. Execution autonomy: they are local transactions, invisible to the global transaction manager — no two-phase commit, no global lock.

One exception, resolved by the allocation method. L5 must check that a vehicle's exporter exists in CONSIGNEE_H, which the global workload alone would place only at S2. Every RoRo registration would have made a remote read, breaking S3's communication autonomy for a local operation. Re-running All Beneficial Sites with L5's 960 reads a day at S3 gives a benefit of 10,148.6 ms/day against a cost of 1,637.4 ms/day: CONSIGNEE_H is now replicated to S3. This also settles what had been the one close call in Section 11.5 — the cost model reached the decision on its own once the local workload was visible to it (Amendment C2).

12.11 Physical design of the local relations

The rule of Section 10 applies unchanged: build an index only if a transaction searches on that key and the file exceeds three blocks.

Relation	Blocks	Access method	Index blocks	Reason
TANK_FARM	1	heap	—	single block
PIPELINE_TRANSFER	49	heap	—	insert-only; nothing searches it
RAIL_RAKE	108	primary index (rake_id)	1	L3 updates rake status by key
RAKE_LOADING	7,569	clustering index (rake_id)	34	Q9 reads by rake; inserts arrive in rake order

Relation	Blocks	Access method	Index blocks	Reason
SEZ_UNIT	3	heap	—	three blocks: a scan is one seek
BOND_TRANSFER	342	heap	—	insert-only
RORO_VEHICLE	5,811	B+ tree (vin), height 3	2,163	L6 probes by vin; arrival order is not vin order
VEHICLE_INSPECTION	3,487	heap	—	insert-only
TIDAL_WINDOW	6	heap	—	six blocks, read as a timetable
LIGHTERAGE	24	primary index (op_id)	1	L7 closes an operation by key

Table 12.7 — Access methods for the local relations

RAKE_LOADING and GATE_MOVE make an instructive pair. Both are large, insert-heavy history relations, yet one gets a clustering index and the other a B+ tree. The difference is arrival order: containers are loaded rake by rake, so RAKE_LOADING is physically in rake_id order and a sparse clustering index of 34 blocks suffices; gate moves arrive in time order, unrelated to cont_id, so GATE_MOVE needs a dense index. The same criterion applied to different local facts gives different answers — which is the point of letting each site design its own internal schema.

Site	Local data	Local index	Share of site storage
S1	50	0	0.03%
S2	8,022	35	2.0%
S3	9,298	2,163	3.8%
S4	30	1	0.02%

Table 12.8 — Storage of the local relations (blocks)

12.12 Effect on the global design

Figure	Before the extensions	After
Relations	19 global	29: 19 global + 10 site-only
Best CONTAINER split (Z)	106,282,875	106,932,375 — same partition
CONSIGNEE_H copies	1 (S2)	2 (S2, S3)
Locality of global accesses	97.35%	97.13% (97.53% counting L1–L8)
Database size	978,665 blocks, 4,009 MB	998,694 blocks, 4,091 MB
Peak disk utilisation	1.61% (S2)	1.81% (S2)

Table 12.9 — What the heterogeneous extension changed

The core design absorbed the extensions with no structural change. That is rule R1 doing its job: because no extension altered a core relation, the only effects on the core are through the workload (Q9 in the affinity matrix, L5 in the allocation), and both were re-evaluated with the same methods as before. Sections 9, 11 and 13 report the updated figures.

12.13 Executable check

To confirm that the four schemas genuinely differ and that the integration is correct, the design was built as four separate SQLite databases — one per port, each with its own schema — and a mediator that attaches all four and answers Q9 through the export views (hetero_demo.py, Appendix C).

Check	Result
The four databases have different relation sets (18, 25, 19, 18 relations)	PASS, all 6 pairs
The 16 relations common to all four have identical columns	PASS
Q9 through the global view equals each value computed directly at the site	PASS, 8 of 8 rows
JNPT: 5 containers loaded onto an open rake — custody unchanged	PASS

Check	Result
JNPT: the rake departs — custody falls by exactly the rake's weight	PASS
Haldia: a barge lands and its 24 containers enter the yard — custody unchanged	PASS
Chennai: the RoRo voyage sails — custody falls by exactly the vehicles aboard	PASS
New Mangalore: 500 kl pumped out at density 0.876 — custody falls by 438 t	PASS

Table 12.10 — Results of the executable prototype

The conservation tests matter most. They move cargo across exactly the boundaries where the four definitions of custody differ, and show that the network total changes only when cargo physically leaves a port.

13. Work Area Space and System Specification

13.1 When does an index beat a scan?

One result governs every query plan. A random block access costs 12.04 ms — seek and latency dominate — while a sequential block costs 0.0403 ms. The ratio is 300 : 1.

Fragment	Blocks @ S2	Full scan	Break-even records
CONTAINER_C	2,637	118.2 ms	3.3
CUSTOMS_DECL	1,890	88.1 ms	2.4
GATE_MOVE	269,532	10,869.6 ms	451.4

Table 13.1 — Index versus scan break-even

The primary indexes on F1 and F8 pay only for single-record access. That is not a contradiction of Section 10 — those indexes exist for U1 and U3, which are precisely single-record operations. It does mean the query optimiser must choose per query: the same fragment is probed by index for U1 and scanned for Q1. GATE_MOVE's break-even is 451 records and Q8 needs about 16, so the B+ tree wins there by a wide margin.

13.2 Work Area Space

Computed per query at S2, the busiest site, as the size of the largest intermediate structure the query must hold in memory — in every case here the build side of a hash join.

Query	Work area	Driven by
Q5	8,164 KB	build side: YARD_SLOT_2 (220,000 × 38 B)
Q3	756 KB	open declarations (10% of 180,000 × 43 B)
Q2	527 KB	hazmat containers (5% of 200,000 × 54 B)
Q7	79 KB	VOYAGE_2 (1,440 × 50 B) + 360 aggregate groups
Q6	64 KB	pipelined scan, ~40-row result
Q8	64 KB	four partial results merged at the originating site
Q1	31 KB	one yard block's 250 slots joined to container rows
Q4	1 KB	today's ~10 voyages joined to VESSEL

Table 13.2 — Work area per query

WAS = 8,164 KB = 7.97 MB, driven by Q5 (yard occupancy per block).

The work area is a choice, not a requirement. Q5 can be executed two ways:

Plan	Work area	Time
Hash join — build on YARD_SLOT_2, stream CONTAINER_C_2 through it	7.97 MB	212 ms
Index nested loop — walk slots in key order, probe CONTAINER_C per slot	0.03 MB	132 min

Table 13.3 — Q5 execution plans

8 MB of buffer buys a 37,398× speed-up. The index-nested-loop plan is feasible — it needs almost no memory, and the slots are already in primary-key order — but it performs 220,000 random probes at 36 ms each. The hash join reads both fragments sequentially, exactly once.

13.3 Memory per site

Site	Work area (×4)	B+ non-leaf	Small idx	Repl. relns	Scan bufs	DBMS	Total
S1	12.0	0.80	0.09	0.07	2.10	64	79.0 MB

Site	Work area (×4)	B+ non-leaf	Small idx	Repl. relns	Scan bufs	DBMS	Total
S2	31.9	2.13	0.33	0.07	2.10	64	100.5 MB
S3	23.9	1.60	0.21	0.07	2.10	64	91.9 MB
S4	12.0	0.80	0.10	0.07	2.10	64	79.0 MB

Table 13.4 — Memory per site (MB), with a concurrency allowance of four heavy queries

The small-index column now includes the pinned indexes of the site-only relations (Section 12.11): the RAKE_LOADING and RAIL_RAKE indexes at S2 and the non-leaf levels of the RORO_VEHICLE B+ tree at S3. The B+ non-leaf column is the buffering result from Section 10.4: pinning 2.13 MB at S2 keeps every internal node of the GATE_MOVE B+ tree resident, so a lookup costs one leaf read plus one data read instead of five random accesses. It is the highest-return memory in the design.

The computed minimum is about 100 MB at S2. A real deployment would provision far more — 8 to 16 GB — not because the design requires it but because spare memory caches hot data and absorbs load spikes. The computed figure is the requirement; the provisioned figure is a recommendation.

13.4 Total system specification

Site	Core data	Core index	Local data	Local index	Replicated	Total blocks	Disk	Memory
S1	104,444	42,085	50	0	16	146,595	600 MB	79 MB
S2	278,507	112,206	8,022	35	1,806	400,576	1,641 MB	101 MB
S3	208,882	84,158	9,298	2,163	446	304,947	1,249 MB	92 MB
S4	104,444	42,085	30	1	16	146,576	600 MB	79 MB
All	696,277	280,534	17,400	2,199	2,284	998,694	4,091 MB	350 MB

Table 13.5 — Total disk and memory

Index overhead is 39.6% of data volume network-wide (40.3% for the core alone), essentially all of it the GATE_MOVE B+ trees. With the availability standbys of Section 11.7, fragment storage doubles to roughly 8 GB.

13.5 Response time per query

Computation time is treated as negligible against I/O — the premise of the topic selection, and justified quantitatively by Section 13.1: even the cheapest operation here is a 12 ms disk access, orders of magnitude above the per-tuple CPU cost of a hash probe.

Query	S1	S2	S3	S4	Notes
Q1 Yard inventory	124	231	189	124	local only
Q2 Hazmat check	107	225	177	107	local only
Q3 Clearance backlog	77	165	130	77	local only
Q4 Vessels at berth	25	25	25	25	local only
Q5 Yard occupancy	95	212	165	95	local only
Q7 Discharge per line	89	156	129	89	local only
Q6 Duty by consignee	217	211	221	226	four sites in parallel
	577	581	579	593	if executed serially
Q8 Demurrage exposure	620	615	625	629	four sites in parallel
	1,604	1,608	1,606	1,620	if executed serially
Q9 Cargo in custody	361	365	355	368	four different sub-queries in parallel (Section 12.9)
	675	679	677	691	if executed serially
U1 Record gate movement	72	72	72	72	local; CONTAINER_G probe + GATE_MOVE

Query	S1	S2	S3	S4	Notes
					insert
U2 Reassign yard slot	108	108	108	108	local
U3 Clear customs	144	144	144	144	local

Table 13.6 — End-to-end response time (ms)

A distributed query processor fans the sub-queries out concurrently, so the response time of Q6, Q8 and Q9 is the slowest site's part plus one round trip, not the sum. Both rows are reported; the serial figures are the honest worst case if the fan-out is not implemented.

Q8 is the slowest query in the system at about 620 ms, dominated by S2's part: a 118 ms scan of CONTAINER_C_2 to find the consignee's containers, then 16 indexed GATE_MOVE lookups at 24 ms each. All updates are local — no two-phase commit, no distributed locking — which is the direct payoff of the fragmentation. U1, at 23,200 a day the most frequent transaction in the system, completes in 72 ms against a single site.

13.6 Disk utilisation

Site	Busy time/day	of which local L1-L8	Utilisation
S1	5.9 min	0.1 min	0.41%
S2	26.0 min	2.8 min	1.81%
S3	17.2 min	2.9 min	1.19%
S4	4.4 min	0.0 min	0.30%

Table 13.7 — Disk utilisation

At 1.81% peak utilisation the system could absorb roughly 55× this workload before the disk saturates. This does not contradict the I/O-bound claim of Section 1.4, and the distinction matters: I/O bound describes where the cost of each operation lies — every response time in Table 13.6 is disk time, with computation negligible — while utilisation describes whether the disk is saturated over time. This workload is I/O bound in character but nowhere near I/O saturated in volume.

The practical consequence is that the binding constraint on this design is response time, not throughput. That is why the effort went into access methods, which cut per-query latency, rather than into partitioning for parallel bandwidth — and why the 8 MB work area of Section 13.2 is worth spending: it buys latency, which is scarce, using memory, which is not.

13.7 Sensitivity and limitations

Every figure in this report follows from the assumptions frozen in Sections 1, 3 and 10.1. Those assumptions are plausible and internally consistent — Appendix B verifies the consistency — but several are estimates, and a reader should know which ones carry a decision.

Assumption	Value	What it decides	If wrong
Query frequencies (Table 3.1)	as tabulated	Every Z value, benefit, cost and utilisation figure	Relative magnitudes matter more than absolutes; the 6–15× replication bar is robust to uniform scaling
Transshipment rate	15%	Migration volume, 6,160/day	Below ~4% the immutable-attribute alternative in Section 9.12 becomes competitive
CONSIGNEE_H update rate	20/day	Replica at S3 (now carried by L5)	With L5 counted the S3 replica holds up to ~124 updates/day; S1 and S4 would need the rate below 3–5/day
Moves per container transit	4	Dwell time, yard population, migration volume	Cross-checked against JNPT throughput in Appendix B; a value of 3 or 5 shifts dwell to 9.1 or 15.2 days
Concurrency allowance	4 heavy queries	Every memory figure in Table 13.4	Memory scales linearly; the work area itself is unaffected
Average seek time	8 ms	The remote/local ratio, and therefore all replication decisions	On SSDs the ratio exceeds 50× and replication becomes attractive — see Section 10.8

Table 13.8 — Assumptions that carry a decision

Two further limitations of the analysis itself. Response times assume a query has the disk to itself; at 1.81% utilisation the queueing error is small, but it is not modelled. And the outputs are quoted at the precision the arithmetic produces ($Z = 106,932,375$) rather than the precision the inputs support — the leading two or three digits are meaningful, the rest are an artefact of exact computation on estimated inputs.

14. Conclusion

The design gives a four-port network four different database schemas built around one shared core. The core is partitioned into 48 fragments and 9 unfragmented relations; each port adds its own site-only relations, ten in all; and a Global-As-View mapping integrates the four schemas for network-wide queries. The whole occupies 4,091 MB, and 97.13% of global accesses are served locally.

Result	Value
Sites	4 (New Mangalore, JNPT, Chennai, Kolkata/Haldia)
Relations after 3NF	29: 19 global + 10 site-only
Local Conceptual Schemas	4, pairwise different (18, 25, 19 and 18 relations); integrated by GAV export views
Fragments	48 (12 families $\times$ 4 sites) + 9 unfragmented relations
Database size	4,091 MB, of which GATE_MOVE is 94.4% of data blocks
Workload locality	97.13% of global accesses local; 97.53% counting local transactions
Best vertical split (CONTAINER)	$Z = 106,932,375$, verified optimal over all 4,094 partitions
Index speed-up on GATE_MOVE	181 $\times$ lookup, at a 17 $\times$ return on maintenance cost
Work area space	7.97 MB, buying a 37,398 $\times$ speed-up on Q5
Value of cost computation	benefit-only allocation would be wrong by 4,970 : 1
Slowest query	Q8 demurrage, 620 ms across four sites in parallel
Most frequent transaction	U1 gate move, 72 ms, entirely local
Peak disk utilisation	1.81% at S2

Table 14.1 — Headline results

Five findings are worth restating. First, the primary key must be excluded from the BEA analysis: including it drives Z below zero at every split point, which is an artefact of an attribute replicated into both fragments by construction rather than a property of the data. Second, BEA searches only contiguous cuts of the clustered order, and exhaustive evaluation of all 4,094 partitions found a split 4% better and considerably more interpretable. Third, and most consequential for the allocation, a remote read in this system costs only 1.15 $\times$ to 1.41 $\times$ a local read, because disk access dominates network latency — so replication must clear a bar of 6 to 15 reads per update, and almost nothing does. Fourth, recorded in Section 10.7: the 1,148 MB index on GATE_MOVE exists only because the schema stores no immutable discharge timestamp — eight bytes on CONTAINER_C would replace it at 287 times lower cost, which is a reminder that optimising an access path is not the same as asking whether the schema should have made the query cheap. Fifth, from Section 12: letting each port design its own extension, under the rule that no extension may alter a core relation, left every core decision intact — and yet the local workload still reached back into the global design, turning the one close allocation call into a clear replica of CONSIGNEE_H at S3. Heterogeneity is not free, but with the right boundary it is cheap.

Appendix A — Amendments

Three corrections were made after the assumptions were frozen (A1, A2, B1). Each was found by a later phase exercising those assumptions, and each landed before the phase that consumed it, so no downstream work was invalidated. Three more (C1–C3) were made when the design was extended with heterogeneous local schemas, after it had been completed; their effect on earlier sections is stated in each case.

#	Found in	Correction	Impact
A1	Logical design	CONTAINER gains voyage_id — Q7 joins to VOYAGE but had no foreign key	Record 72 → 76 B; attribute set extends to A13; strengthens query class C
A2	Logical design	PORT is an entity from the EER, not a 3NF artefact	3NF work reduced to HS_CODE_MASTER
B1	System spec	YARD_SLOT 60,000 → 550,000 — only 12% of resident containers could have occupied a slot	F4 558 → 5,105 blocks (+18.6 MB); WAS rises to 7.97 MB
C1	Heterogeneous extension	Ten site-only relations, local transactions L1–L8 and the integrated query Q9 added (Section 12)	Split unchanged, Z → 106,932,375; locality 97.13%; +19,599 blocks
C2	Heterogeneous extension	CONSIGNEE_H replicated to S3 — L5 reads it 960 times a day	+430 blocks at S3; S3 keeps communication autonomy for L5
C3	Heterogeneous extension	GATE_MOVE.cont_id declared a logical reference — history outlives the CONTAINER row	Table 7.2 note; no change to storage or timings

Table A.1 — Amendments to the frozen assumptions

Amendment B1 was settled by a cross-check against real throughput: JNPT handles 6,000,000 TEU a year, and at the 12.2-day average dwell derived in Section 9.12 that puts 200,548 containers in the yard — against the 200,000 assigned to S2. CONTAINER was right and YARD_SLOT was under-specified by roughly nine times.

Appendix B — Verification

Every derived figure was recomputed from the frozen assumptions by an independent script and compared against what each section claims — 87 automated checks, 44 of them added for the heterogeneous extension.

Category	Checks	Result
Record sizes recomputed from attribute lists	19	19 PASS
Vertical fragmentation completeness, disjointness, overhead	3	3 PASS
Per-site tuple counts sum to relation totals	10	10 PASS
Derived constants (Tr, Td, Tp, random access)	5	5 PASS
Internal coherence of the workload assumptions	6	6 PASS
Local relations: record sizes, blocks, index blocks, RoRo B+ tree	31	31 PASS
Local Conceptual Schemas pairwise different (6 pairs)	6	6 PASS
Executable prototype: common core identical, four schemas different, uniform export shape	3	3 PASS
Storage, GATE_MOVE share and locality restated with the extension	4	4 PASS
Total	87	87 PASS

Table B.1 — Verification results

The coherence checks establish that the assumptions are a consistent picture rather than independent guesses:

60,000,000 gate moves/yr / 4 per transit = 15,000,000 transits/yr = 41,096/day
 500,000 resident containers / 41,096 per day = 12.2 days average dwell

- 12.2 days exceeds the 5-day free period, so Q8 has a real population to measure.
- JNPT at 6M TEU/yr and 12.2 days dwell gives 200,000 containers in yard, against the 200,000 assigned to S2 — agreement to 0.0%.
- 550,000 yard slots exceed 500,000 resident containers — 91% occupancy, leaving working headroom.
- GATE_MOVE is 96.8% of the core data blocks and 94.4% including the site-only relations, confirming the I/O-bound topic selection against the finished design.

One further finding: splitting GATE_MOVE into four fragments costs 2 extra blocks (8 KB) against keeping it whole, because each fragment rounds up to a whole block independently. Negligible, but it is a genuine cost of fragmentation.

What these checks do not cover. They verify that the numbers agree with one another — that record sizes sum correctly, tuple counts reconcile, and derived constants match the formulas. They cannot establish that the design is a good answer to the problem. Whether the right queries were chosen, whether an attribute is missing from a relation, or whether an access path should have existed at all are questions no arithmetic check can reach. Section 10.7 records one such finding, identified by reasoning about the schema rather than by any of the 87 checks, all of which it passes. Amendment C3 is another: the GATE_MOVE foreign key could not be enforced as declared, and every arithmetic check passed regardless.

Appendix C — Computation

The matrices, split-quality tables, allocation figures, figures and verification results were produced by the following Python scripts, supplied with this report:

Script	Produces
frag.py	Attribute affinity matrices, BEA clustering, split quality, exhaustive partition search
physical.py	Blocking, index structures, access-method comparison, timing tables
alloc.py	Benefit and cost per fragment per site, All Beneficial Sites decisions
phase6.py	Work area, memory, response times, disk utilisation
verify.py	The 87 cross-section consistency checks of Appendix B
local_schemas.py	Site-only relations, their physical design, local transactions and Q9 sub-query costs (imported by alloc.py, phase6.py, verify.py)
hetero_demo.py	Four SQLite databases with different schemas, the export views, the mediator, and the checks of Table 12.10
eer.py, figs.py, hetero_figs.py	Figures 1–6

Table C.1 — Supplied computation

The core of the split-quality evaluation, for reference:

```
def split_quality(order, use, n):
    rows = []
    for cut in range(1, n):
        TA, TB = set(order[:cut]), set(order[cut:])
        TQ = [q for q, c in use.items() if {i-1 for i in c} <= TA]
        BQ = [q for q, c in use.items() if {i-1 for i in c} <= TB]
        OQ = [q for q, c in use.items()
              if not ({i-1 for i in c} <= TA or {i-1 for i in c} <= TB)]
        CTQ, CBQ, COQ = (sum(TOT[q] for q in g) for g in (TQ, BQ, OQ))
        rows.append(dict(cut=cut, CTQ=CTQ, CBQ=CBQ, COQ=COQ,
                        Z=CTQ*CBQ - COQ*COQ))
    return rows
```